

Containers, Kubernetes and Google Cloud

Mete Atamel

Developer Advocate for Google
Cloud



Mete Atamel

Developer Advocate for Google Cloud

@meteatame

| atamel@google.com

meteatamel.wordpress.co

m

Please send
talk feedback:
bit.ly/atamel



Software development is
HARD!
And it is not getting any
easier...

A collection of vintage computer storage media is shown. In the foreground, a red 5.25-inch floppy disk is prominent, with a red arrow pointing to its center hole. The label on the floppy disk reads "boedem", "FLOPPY DISK", and "5,25 HD,DS". To its left is a dark grey 5.25-inch hard disk with a label that includes "EC-5274", "SSA", and "СДЕЛАНО В БОЛГАРИИ". To the right is a blue 3.5-inch floppy disk. The background is a plain, light-colored surface.

This is all I had to care...

Life was good :-)

A lot happened since then

Interne
t

Database
s

App
Servers

Firewalls

Object
Oriented
Programming

DevOp
s

Web Servers
Microservice
s

Version
Control

Virtual
Machine
s

The
Monolith

Cloud
Computing

Cachin
g

Big
Data

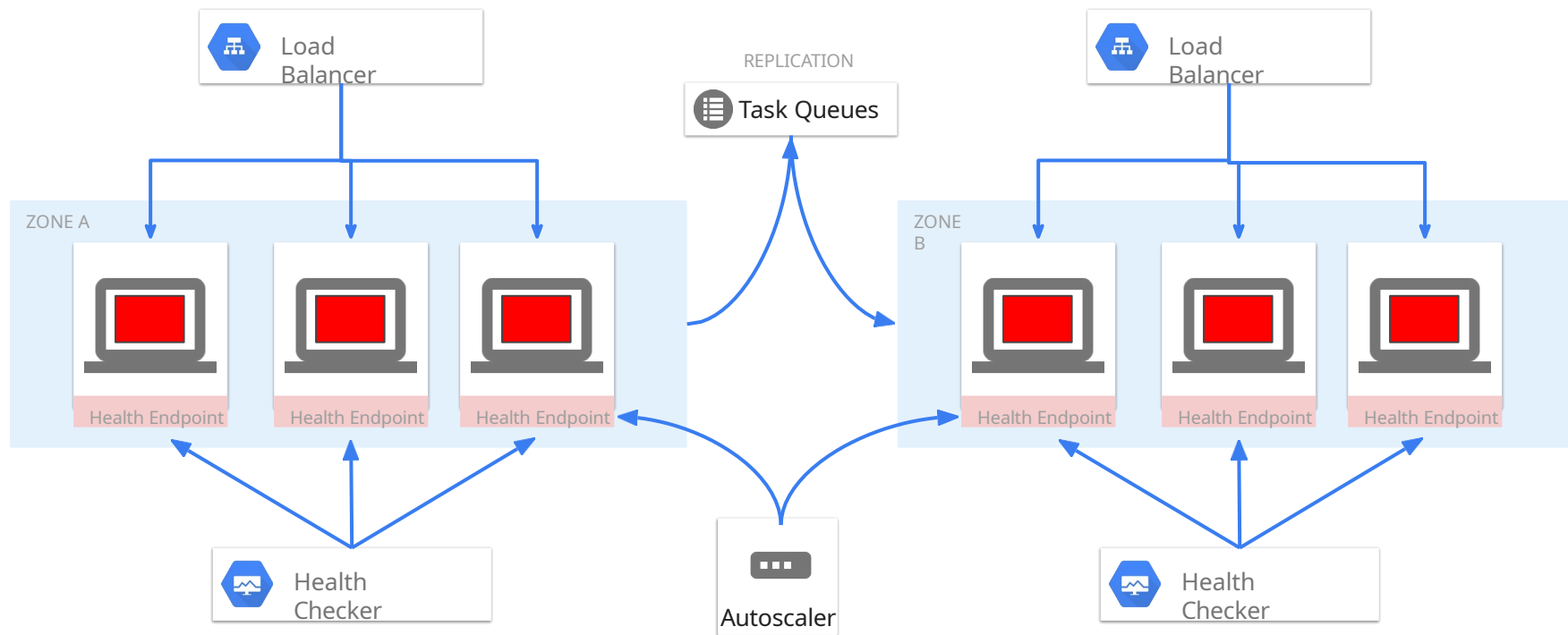
Mobil
e

Machin
e
Learnin
g

IoT



Nowadays



We haven't even talked about

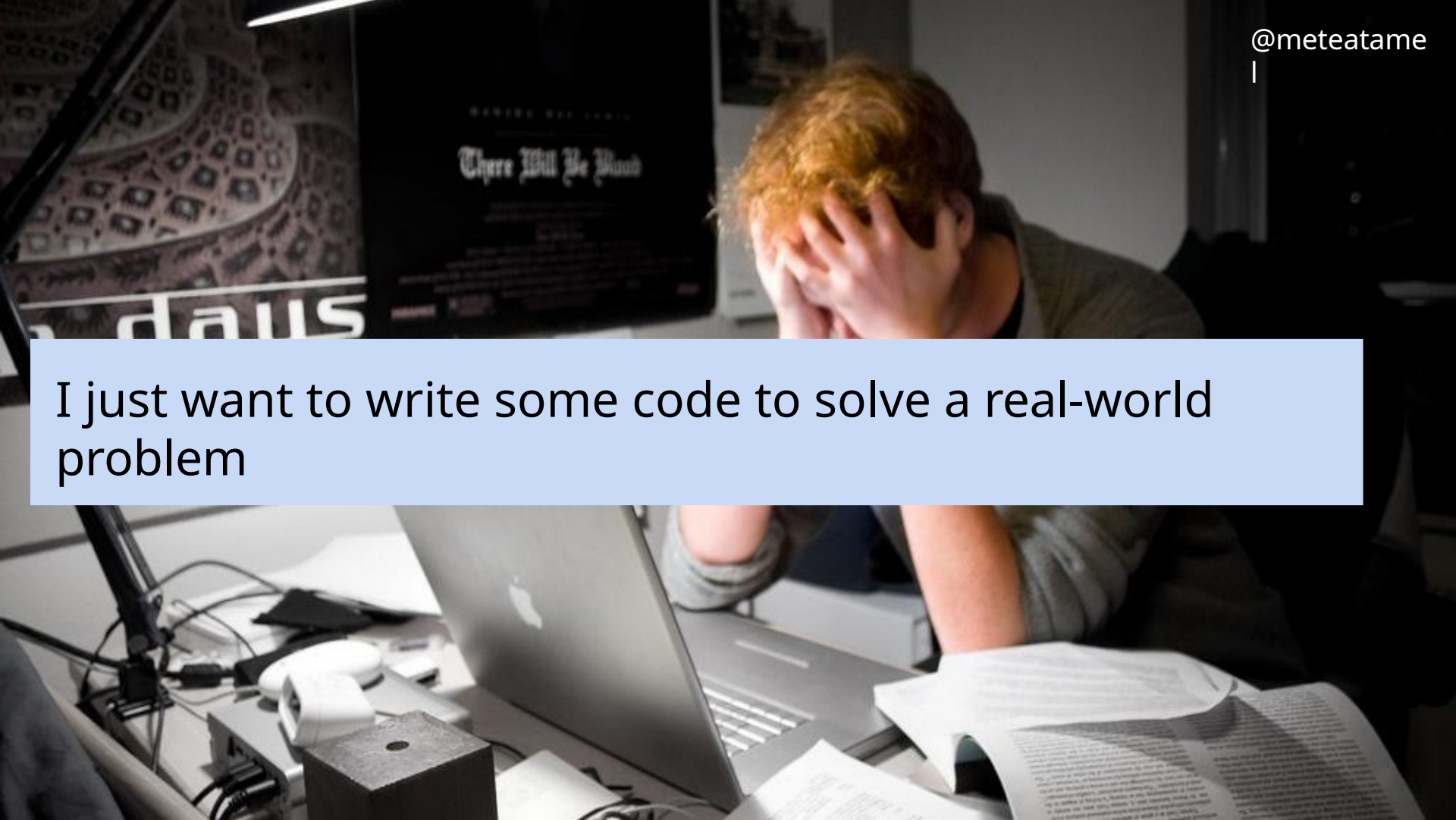
Maintaining code in **different** languages on **different** types of machines

Rolling out the **new version** of your code reliably

Rolling back to the **old version** if something goes

wrong Managing **configuration** and **secrets**

Managing **scripts** that need to run on each machine

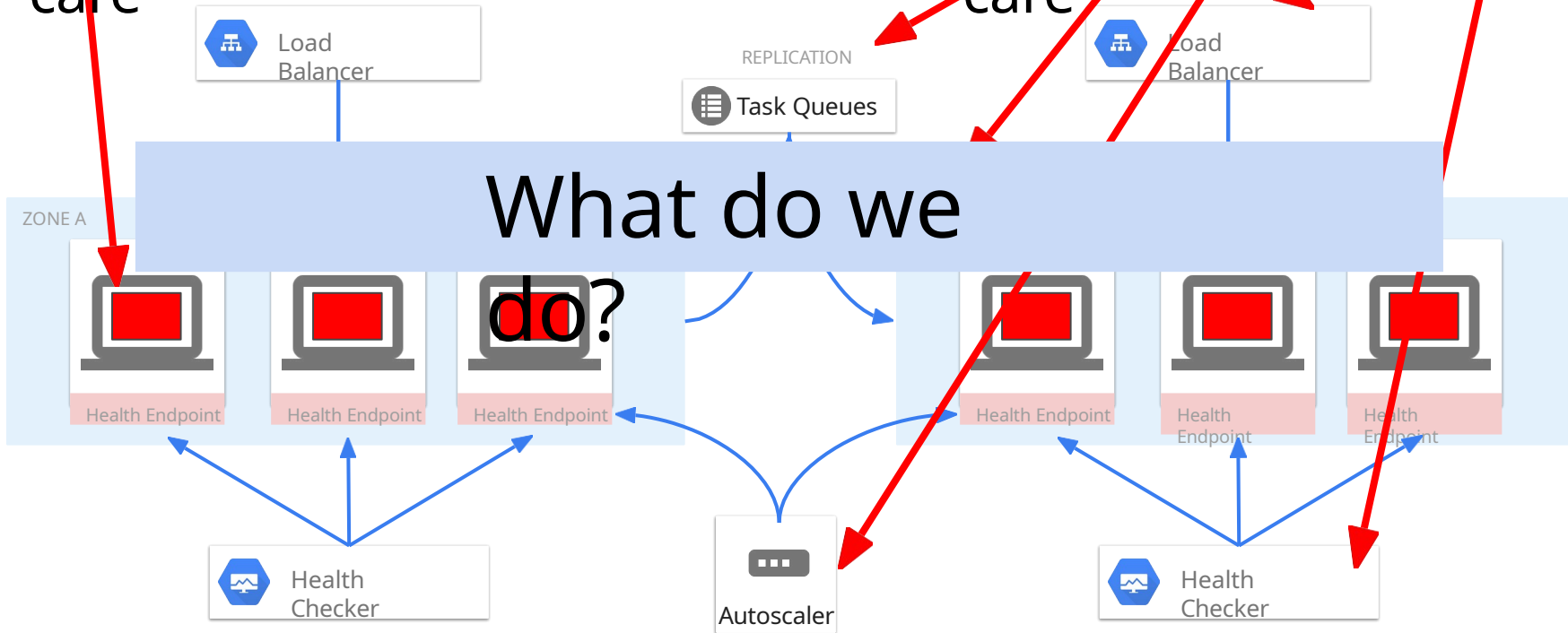
A person with red hair is sitting at a desk, covering their face with their hands in a gesture of frustration or despair. They are surrounded by multiple computer monitors. One monitor in the background displays the title "There Will Be Blood" in a stylized font. Another monitor to the left shows a patterned image. The desk is cluttered with papers, a laptop, and various cables. The scene is dimly lit, with light coming from the monitors and a desk lamp.

I just want to write some code to solve a real-world problem

The reality

This is all I **want** to care

This is all I **have** to care



In the good old days

@meteatame
|

Write your code, pass it to QA for testing, let operations team run it...



Nowadays, it is your problem

What do we do?

DevOps
;-)

You can write your code in **any** language and
run
anywhere exactly the **same** way

Your app is optimally deployed **somewhere**
and managed by **someone**. It just
works!

There are no machines. All resources are
automatically provisioned on demand

Docker + Kubernetes + Cloud

Write your code in any language and run it anywhere exactly the same way

- ~ Containers (eg. Docker, Rkt)

Your app is optimally deployed and managed

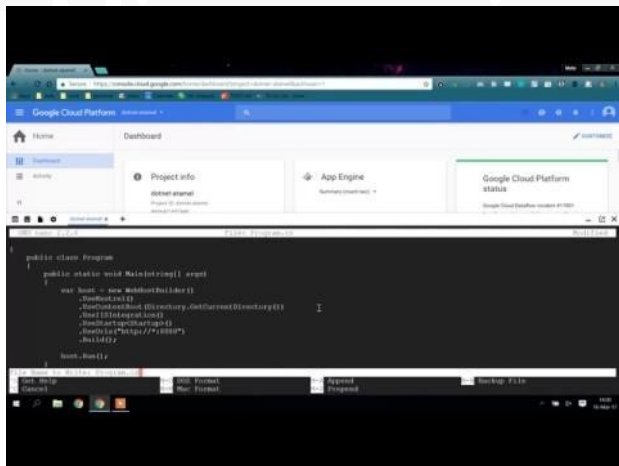
- ~ Container Management Platforms (eg. Kubernetes, Docker Swarm, Mesos)

All the resources needed for your app is automatically provisioned per demand

- ~ Cloud Providers (eg. Google Cloud, AWS, Azure)



Demo: Simple

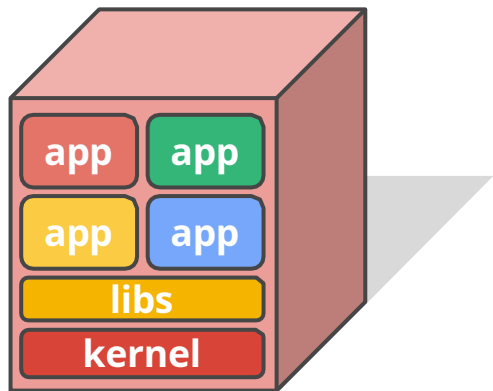


Container



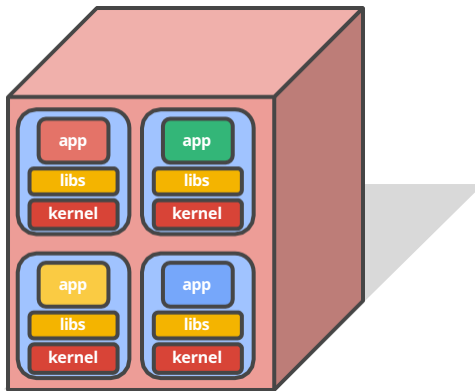
Why containers?

@meteame
|



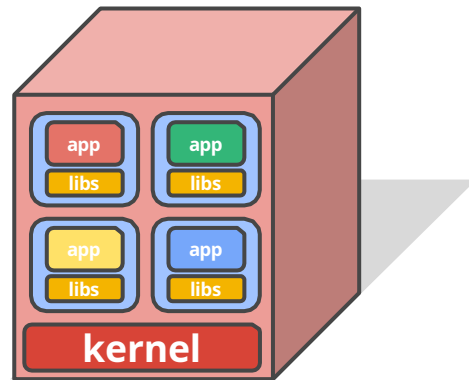
Physical Machine

- ✗ No isolation
- ✗ Common libs
- ✗ Highly coupled Apps & OS



Virtual Machines

- ✓ Isolation
- ✓ No Common Libs
- ✗ Expensive and Inefficient
- ✗ Hard to manage



Containers

- ✓ Isolation
- ✓ No Common Libs
- ✓ Less overhead
- ✗ Less Dependency on Host OS

What is a container?

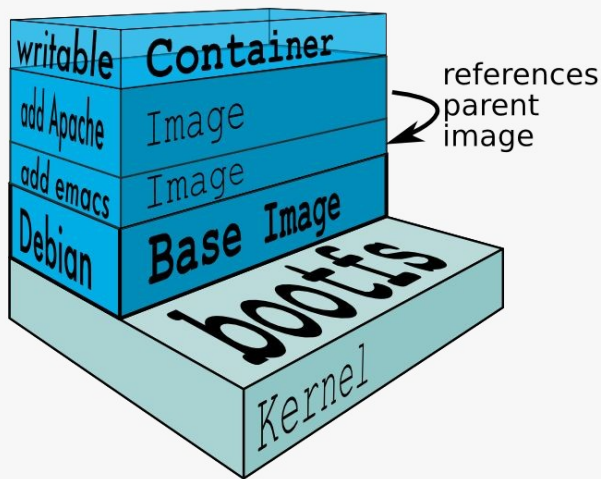
@meteatame
|

A **lightweight** way to virtualize applications

Linux (or Windows) processes

Lightweight
Hermetically
sealed Isolated

Easily
deployable
Introspectable
Composable





@meateatme

Google has been
developing and using
containers to manage our
applications for **over 12
years.**

Images by Connie
Zhou

Everything at Google runs in containers

Gmail, Web Search,
Maps, ... MapReduce,
batch, ...

GFS, Colossus, ...

Even **Google's Cloud Platform**: our
VMs run in containers!

We launch over **2 billion**
containers **per week**

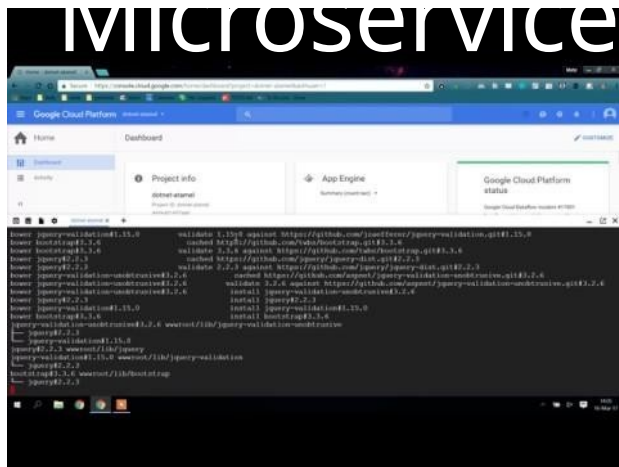


@meteame

Shipping Containers At Clyde, by Steve Gibson

Demo: Containerised

Microservice



Containers help to create a lightweight and consistent environment for apps

But you still need to answer these questions:

- Who takes care of **redundancy**?
- Who takes care of **resiliency**?
- Who **scales up/down** your app?
- Who and how a **new version** of your app gets deployed?
- Who rolls back to a **previous version** if something goes wrong?
- Etc. etc. etc.

Kubernetes



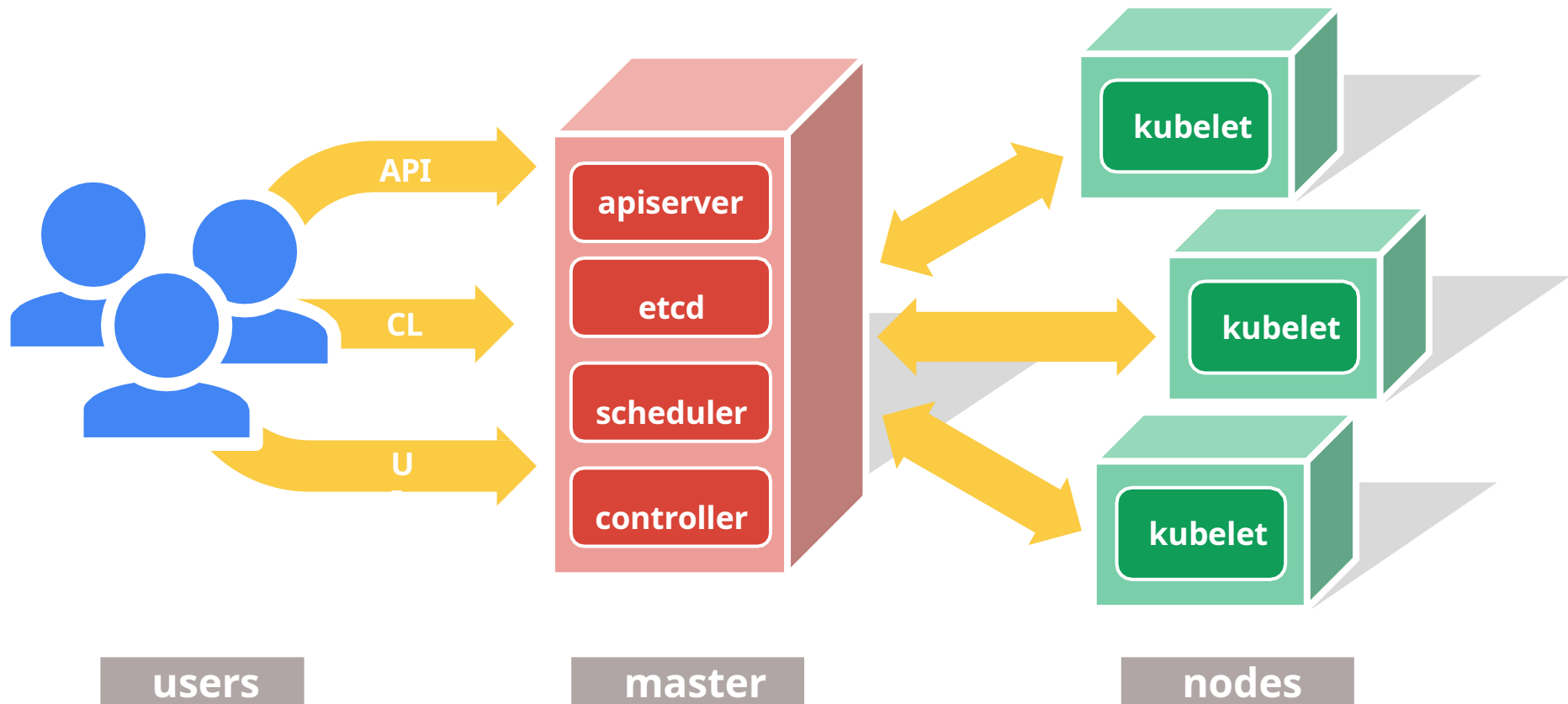
Greek for *“Helmsman”*; also the root of the words *“governor”* and *“cybernetic”*

- Manages container clusters
- Inspired and informed by Google’s experiences and internal systems (borg)
- Supports multiple cloud and bare-metal environments
- Supports multiple container runtimes
- **100% Open source**, written in Go



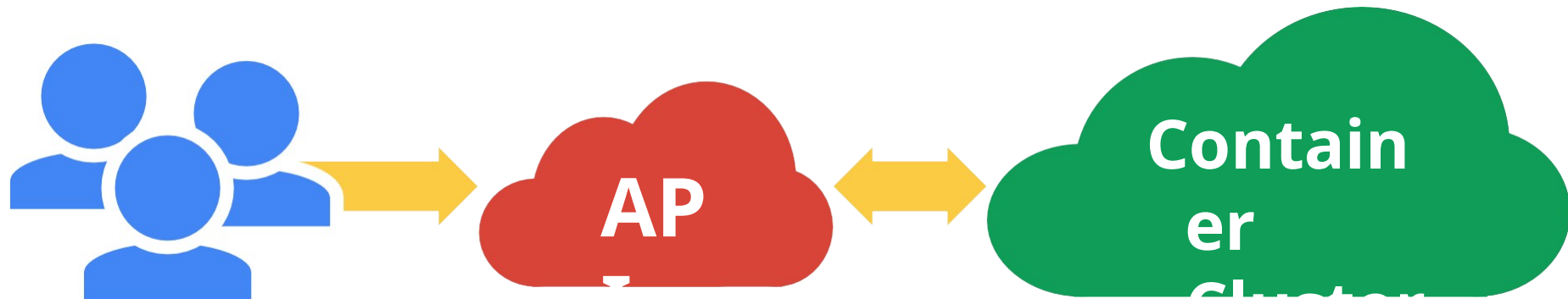
The 10000 foot view

@meteame
|



All you really care

@meteatame
|



1. Setting up the cluster

- Choose a cloud: GCE, AWS, Azure, Rackspace, on-premises, ...
- Choose a node OS: CoreOS, Atomic, RHEL, Debian, CentOS, Ubuntu, ...
- Provision machines: Boot VMs, install and run kube components, ...
- Configure networking: IP ranges for Pods, Services, SDN, ...
- Start cluster services: DNS, logging, monitoring, ...
- Manage nodes: kernel upgrades, OS updates, hardware failures...

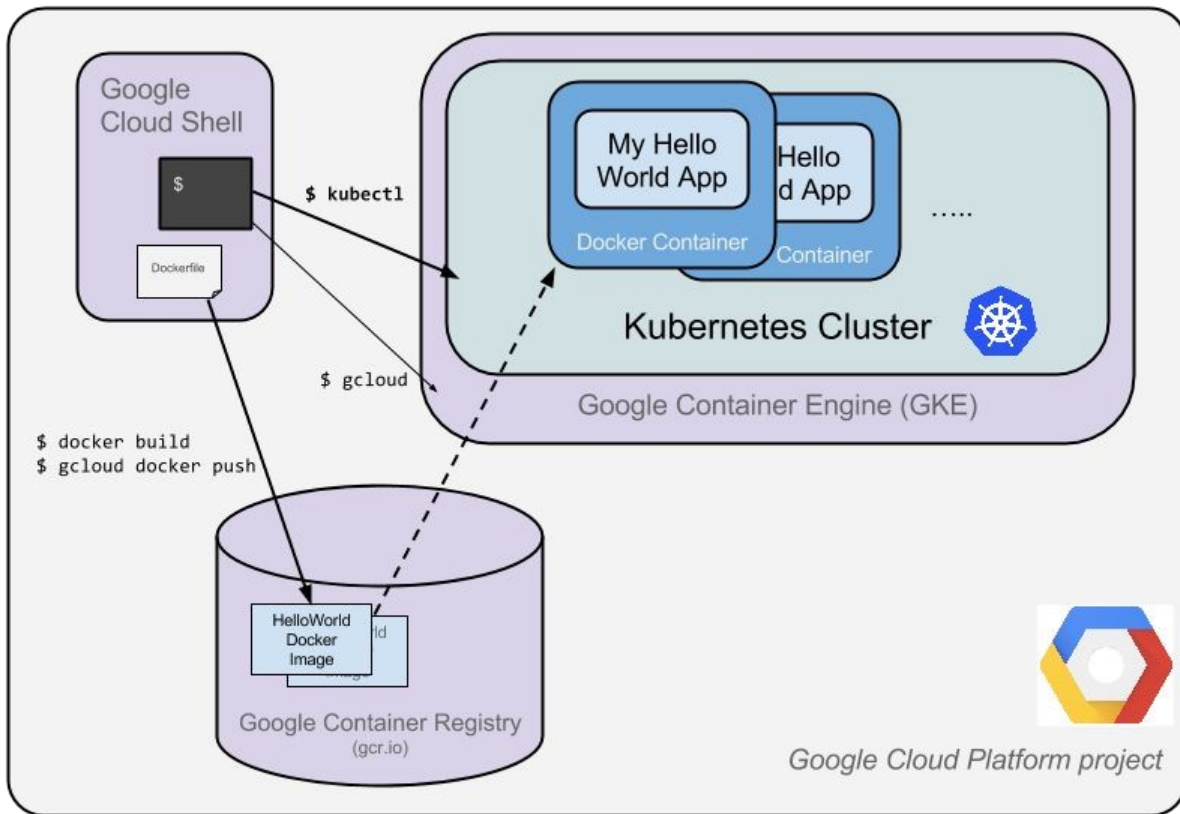
Not the easy or fun part, but unavoidable

This is where things like **Google Container Engine (GKE)** really help

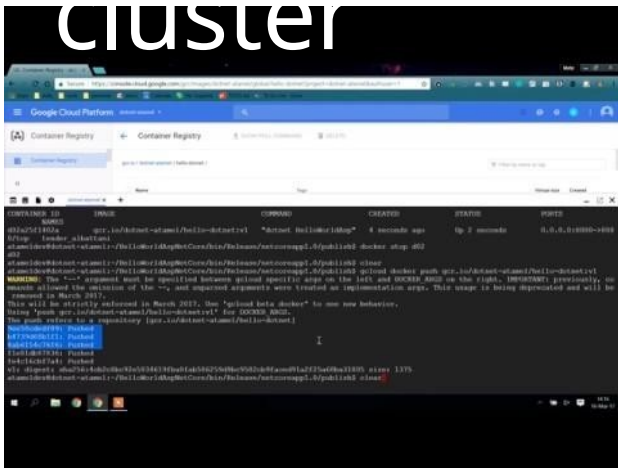


Kubernetes cluster on GKE

@meteatame
|



Demo: Create Kubernetes



2. Using the cluster

- Run Pods & Containers
- Replica Sets
- Services
- Volumes

This is the fun part!

A distinct set of problems from cluster setup and management Don't make developers deal with cluster administration!

 Accelerate development by focusing on the applications,

Kubernetes Building

Kubernetes Terminology

Deployment

ReplicaSet

DaemonSet

Pod

Liveness Probe

Job

Volume

Readiness Probe

StatefulSet

Label

Service

ConfigMap

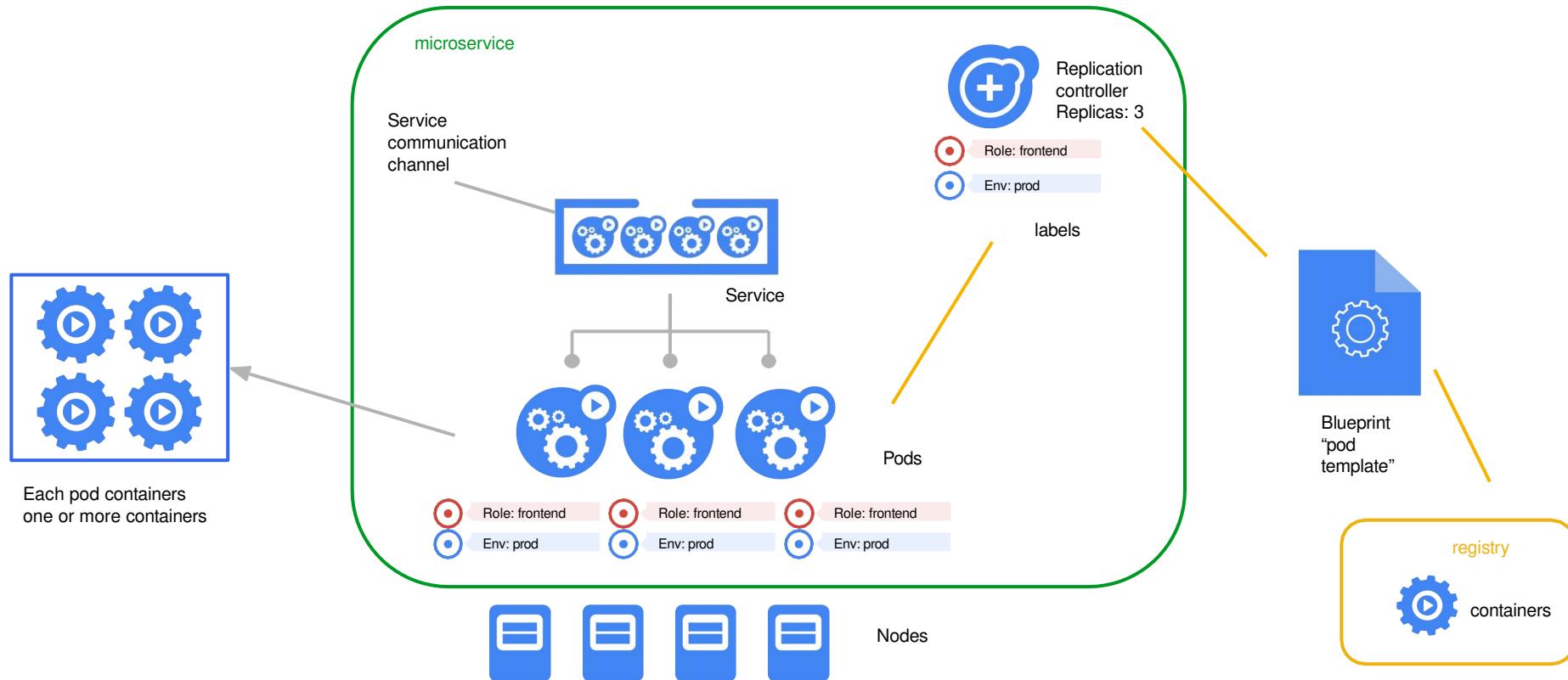
Selector

Secret



Container cluster

@meteatame
|



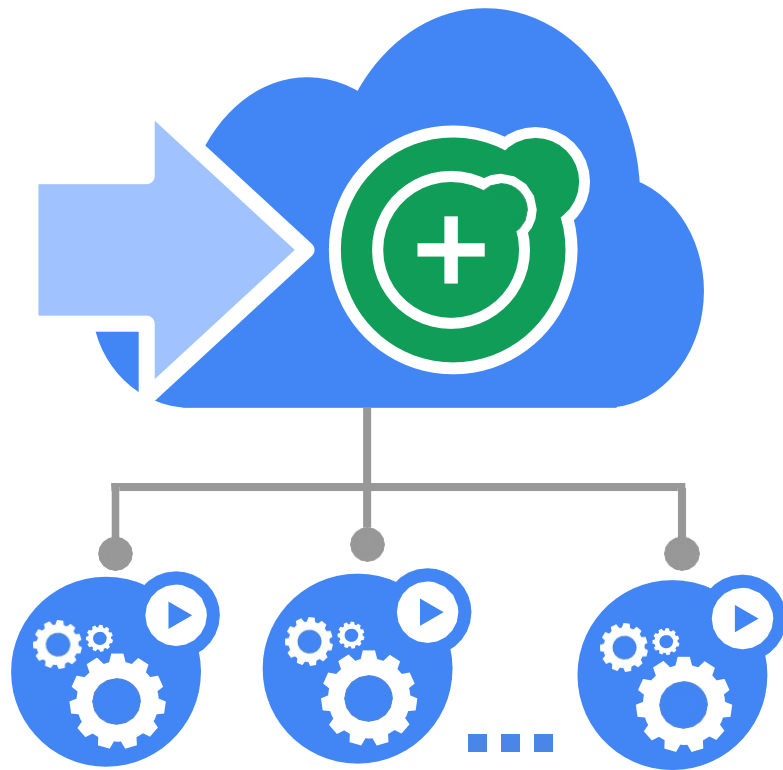
Deployments: where everything

A Deployment provides declarative updates for Pods and Replica Sets

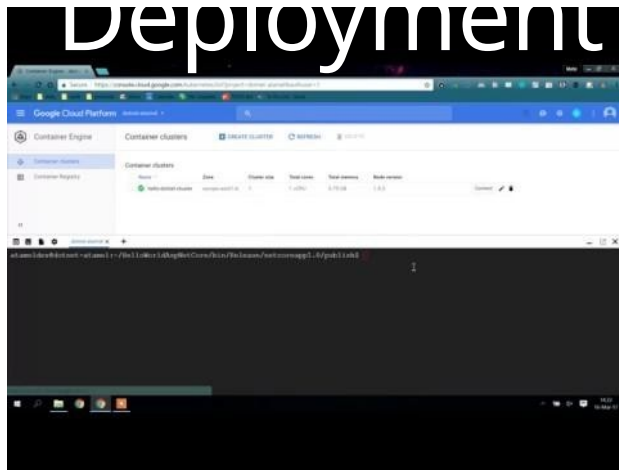
Describe the desired state and the Deployment controller will change the actual state to the desired state at a controlled rate for you.

Deployment manages replica changes for you
updates are configurable, done server-side

- `kubectl edit` or `kubectl apply`



Demo: Create Deployment



Kubernetes Terminology

Deployment

ReplicaSet

DaemonSet

Pod

Liveness Probe

Job

Volume

Readiness Probe

StatefulSet

Label

Service

ConfigMap

Selector

Secret



Pods and Volumes: Atom of



Small group of containers & volumes

Tightly coupled

The atom of scheduling &

placement. Shares & namespace
localhost

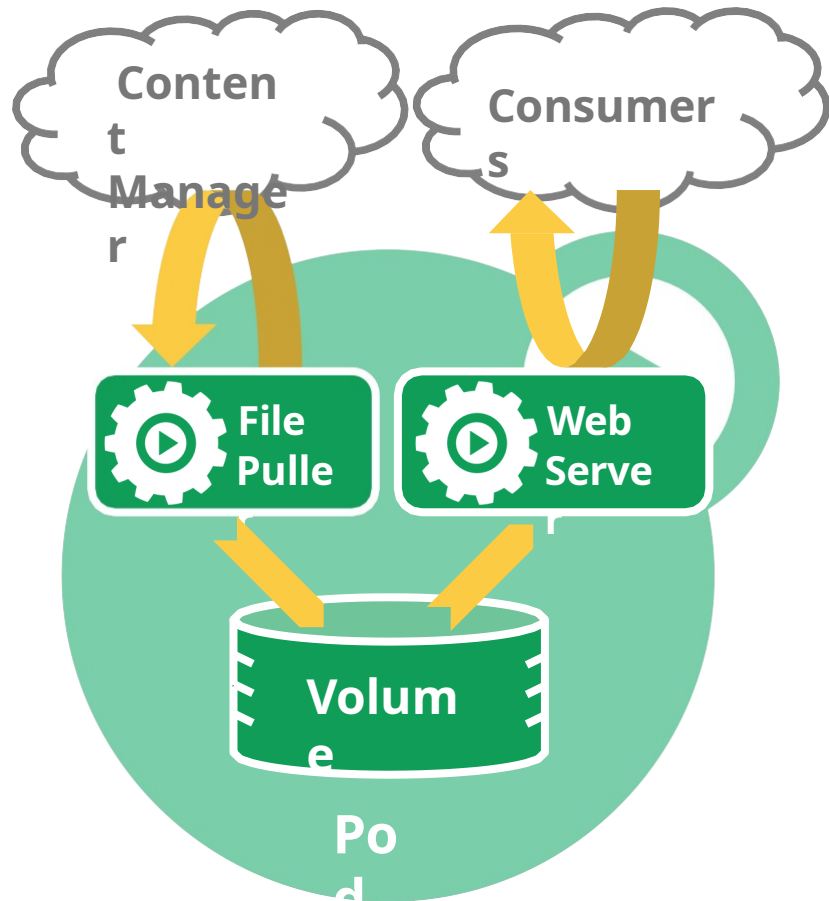
- share IPC, etc.

Managed

lifecycle. Tied to a node, restart in place

- can die, cannot be reborn with same ID

Example: data puller & web server



Pod-scoped storage

Support many types of volume plugins

- Empty dir (and tmpfs)
- Host path
- Git repository
- GCE Persistent Disk
- AWS Elastic Block Store
- vSphere
- GlusterFS
- Ceph File and RBD
- Cinder
- FibreChannel
- Secret, ConfigMap, DownwardAPI
- Flex (exec a binary)
- ...



Kubernetes Terminology

Deployment

ReplicaSet

DaemonSet

Pod

Liveness Probe

Job

Volume

Readiness Probe

StatefulSet

Label

Service

ConfigMap

Selector

Secret



Labels & Selectors:



Arbitrary metadata

Attached to any API

object Generally

represent **identity**

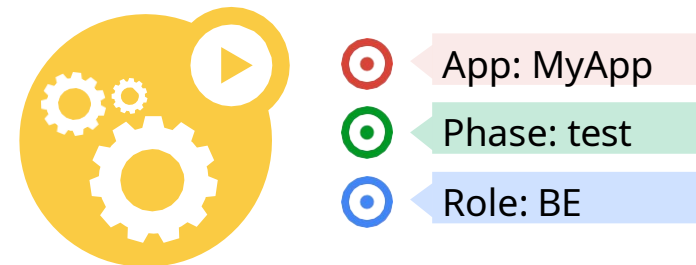
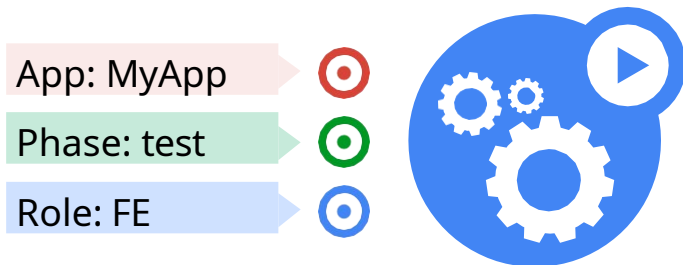
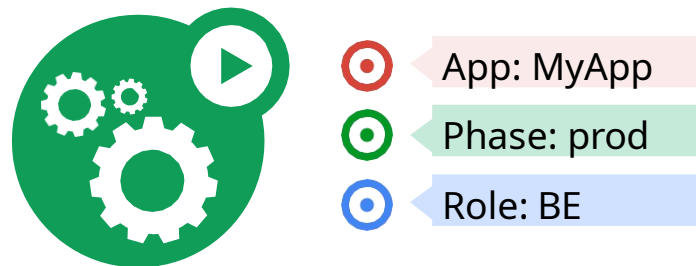
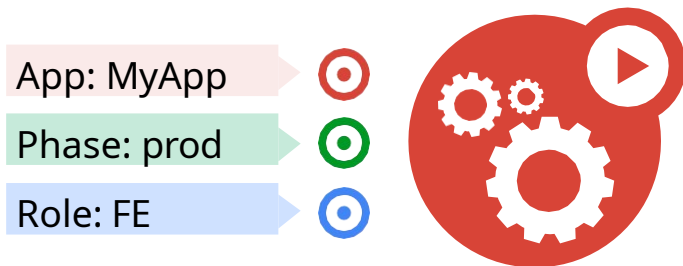
Queryable by **selectors**

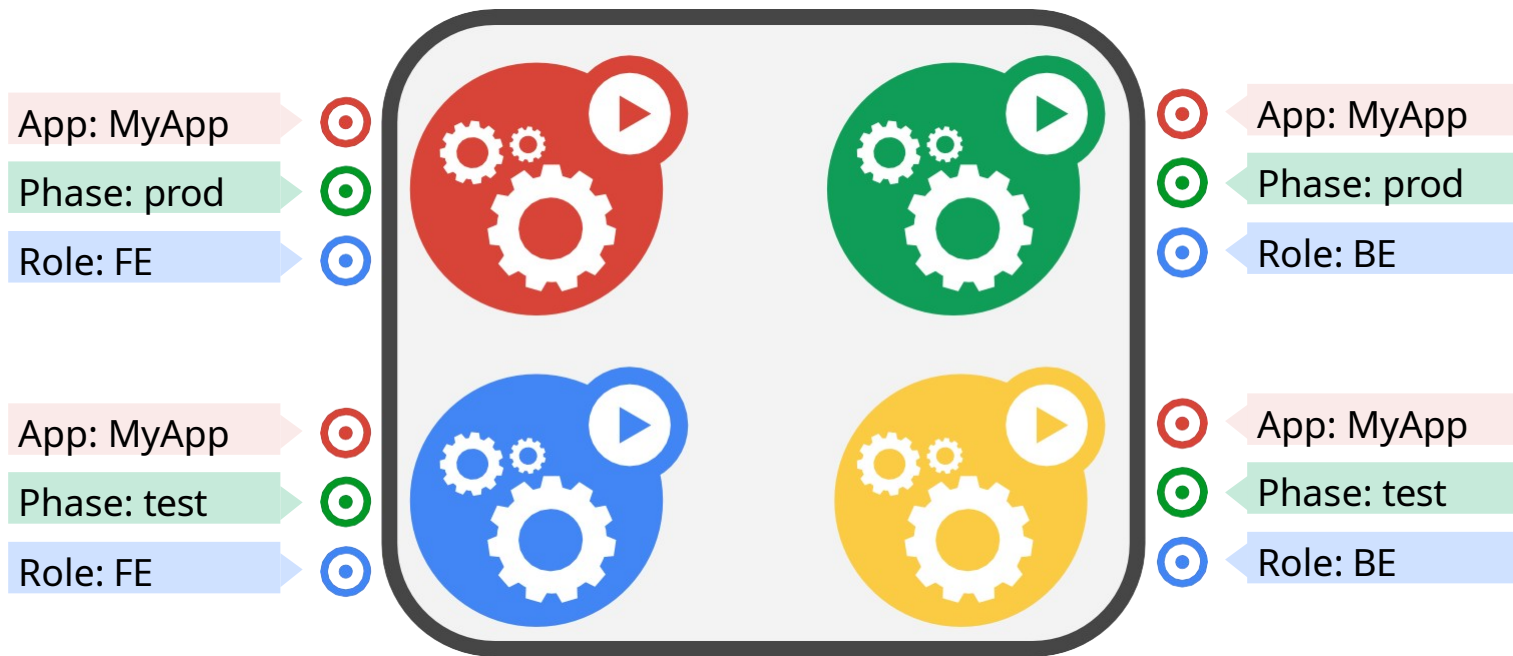
- think SQL *'select ... where ...'*

The **only** grouping mechanism

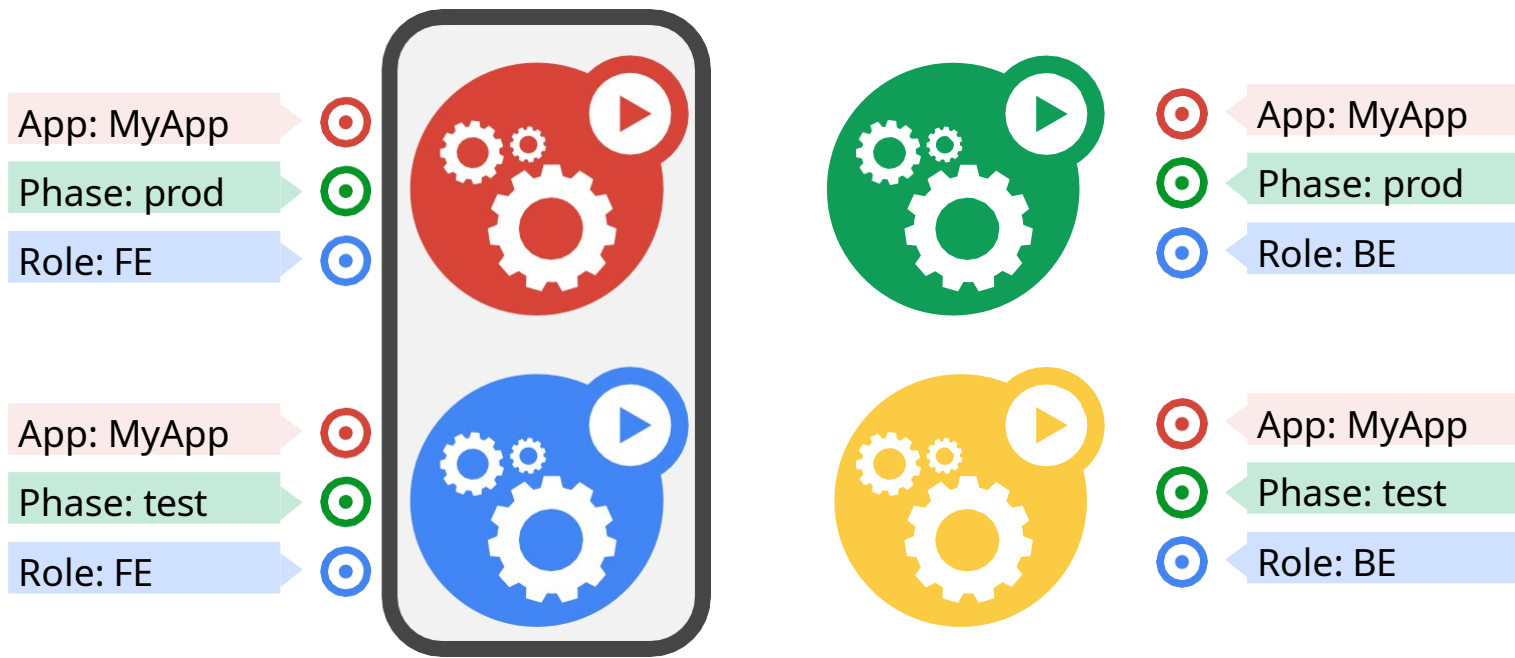
- pods under a ReplicationController
- pods in a Service
- capabilities of a node



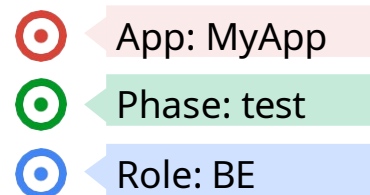
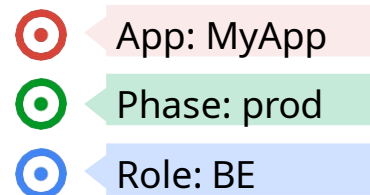
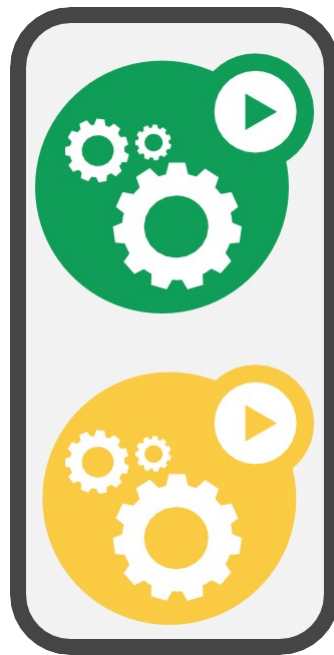
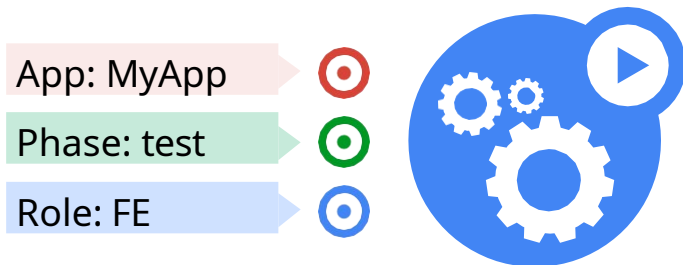
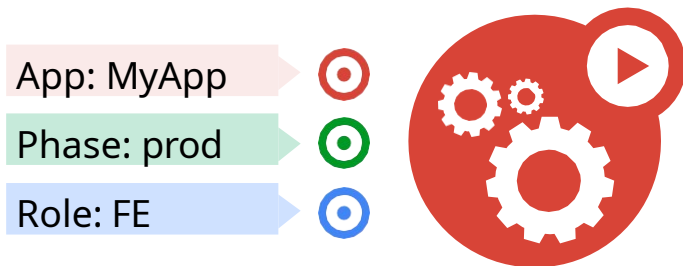




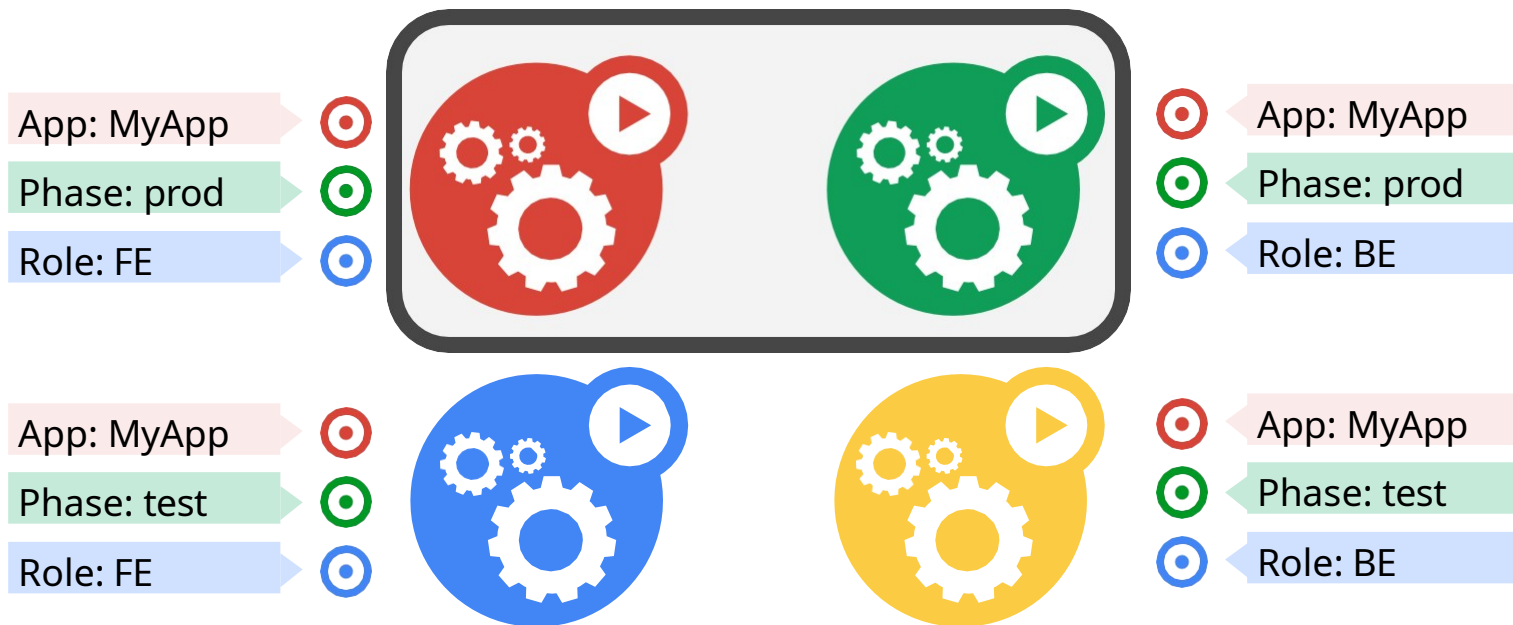
App =
MyApp



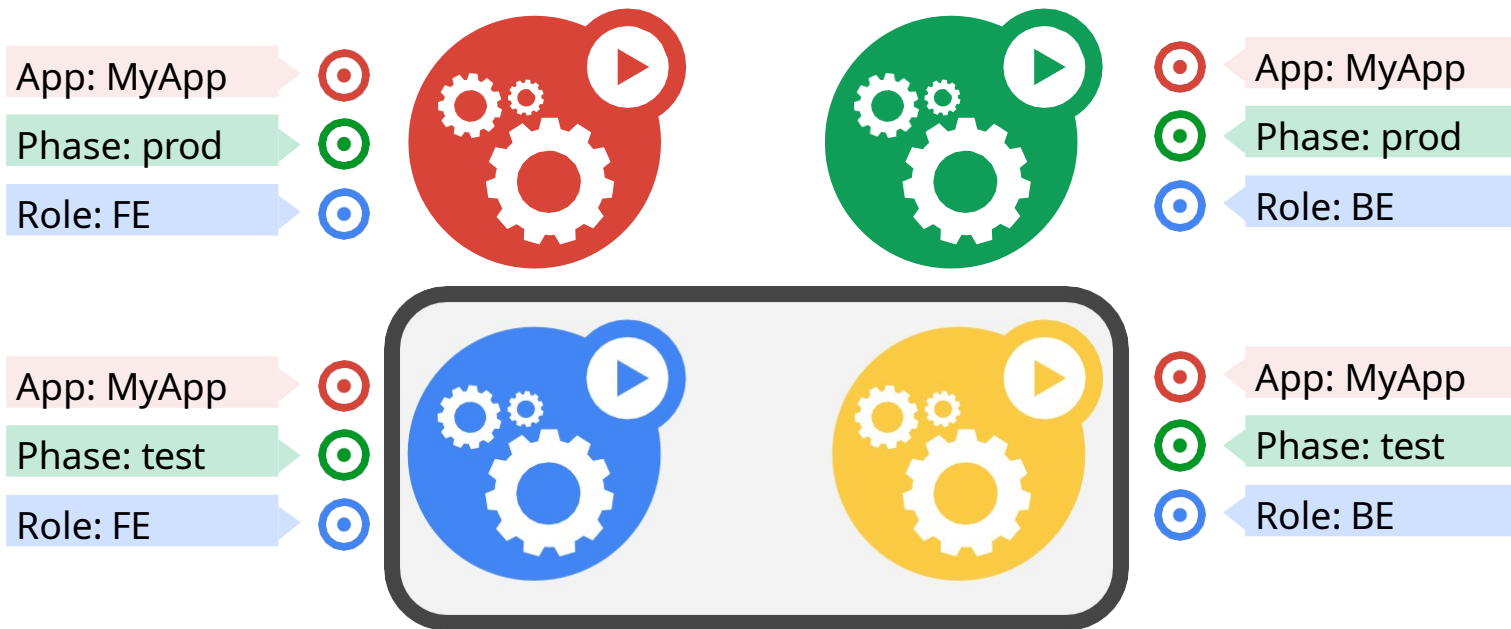
**App = MyApp, Role =
FE**



**App = MyApp, Role =
BE**



**App = MyApp, Phase =
prod**



**App = MyApp, Phase =
test**

Kubernetes Terminology

Deployment

ReplicaSet

DaemonSet

Pod

Liveness Probe

Job

Volume

Readiness Probe

StatefulSet

Label

Service

ConfigMap

Selector

Secret



Resiliency &



ReplicaSets: Big brother

@meteatame
|

A simple control loop

Runs out-of-process wrt API server

One job: ensure N copies of a pod
grouped by a selector

- too few? start some
- too many? kill some

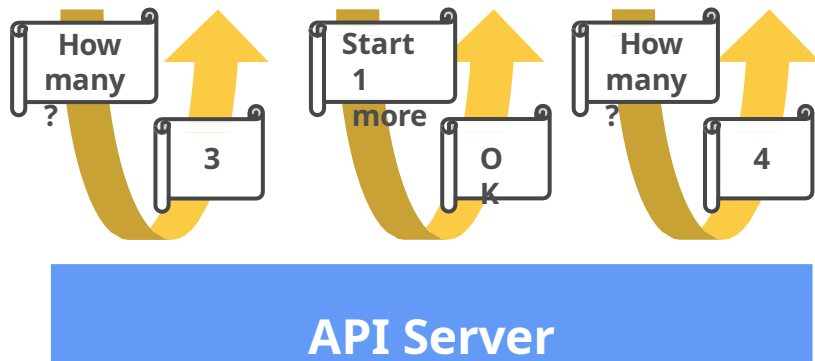
Layered on top of the public Pod API

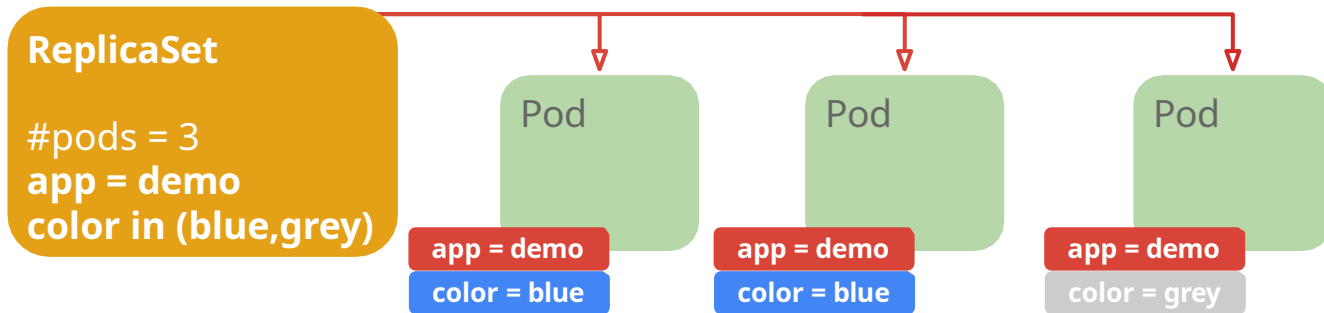
Replicated pods are **fungible**

- No implied order or identity

ReplicaSet

```
t- name = "my-rc"
- selector = {"App":
  "MyApp"}
- template = { ... }
```





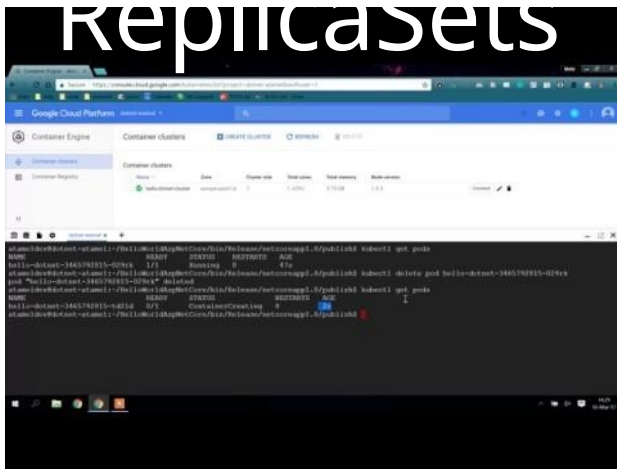
Behavior

- Keeps Pods running
- Gives direct control of Pod #s
- Grouped by Label Selector

Benefits

- Recreates Pods, maintains desired state
- Fine-grained control for scaling
- Standard grouping semantics

Demo: ReplicaSets



Kubernetes Health



Health Check Philosophy



It's **your** responsibility to let Kubernetes know whether your app is healthy or not!

Liveness Probes

Liveness Probes make sure your application is running

livenessProbe:

an http

probe

httpGet:

path:

/healthz

port: 8080

initialDelaySe

conds: 15

timeoutSecon

ds: 1

wait 15 seconds after pod is started to check for health
wait 1 second for a response to health check



Readiness Probes

Readiness probes make sure your application is ready to serve traffic

```
readinessProbe:
```

```
  # an http
```

```
  probe
```

```
  httpGet:
```

```
    path:
```

```
    /readiness
```

```
    port: 8080
```

```
  initialDelaySeconds: 20 # wait 20 seconds after pod is started to check for
```

```
  health timeoutSeconds: 5 # wait 5 second for a response to health check
```

Kubernetes Terminology

Deployment

ReplicaSet

DaemonSet

Pod

Liveness Probe

Job

Volume

Readiness Probe

StatefulSet

Label

Service

ConfigMap

Selector

Secret



Service



A logical grouping of pods that perform the same function (the Service's endpoints)

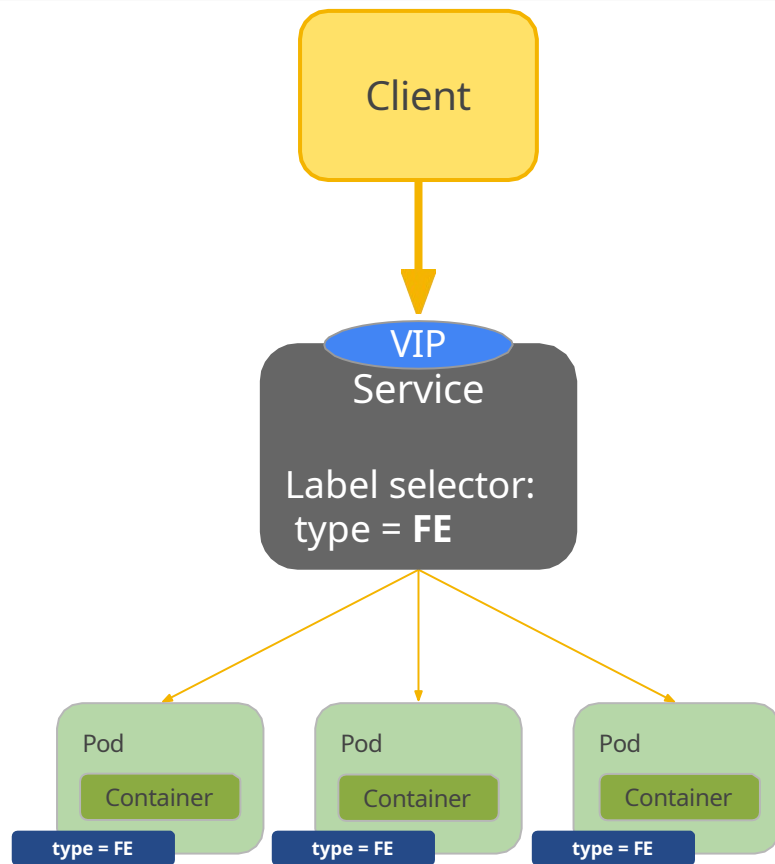
- grouped by label selector

Load balances incoming requests across constituent pods

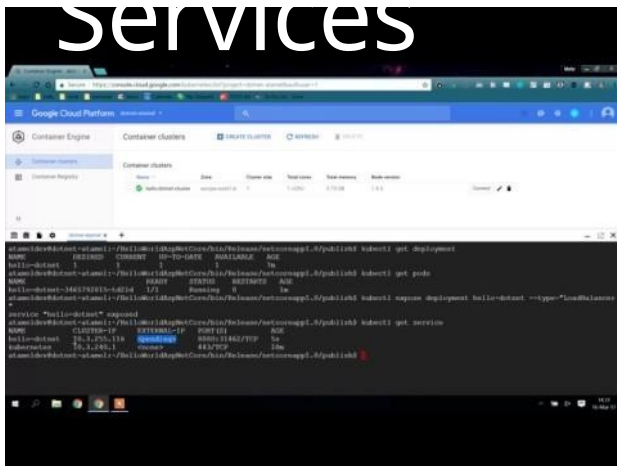
Choice of pod is random but supports session affinity (ClientIP)

Gets a **stable** virtual IP and port

- also a DNS name

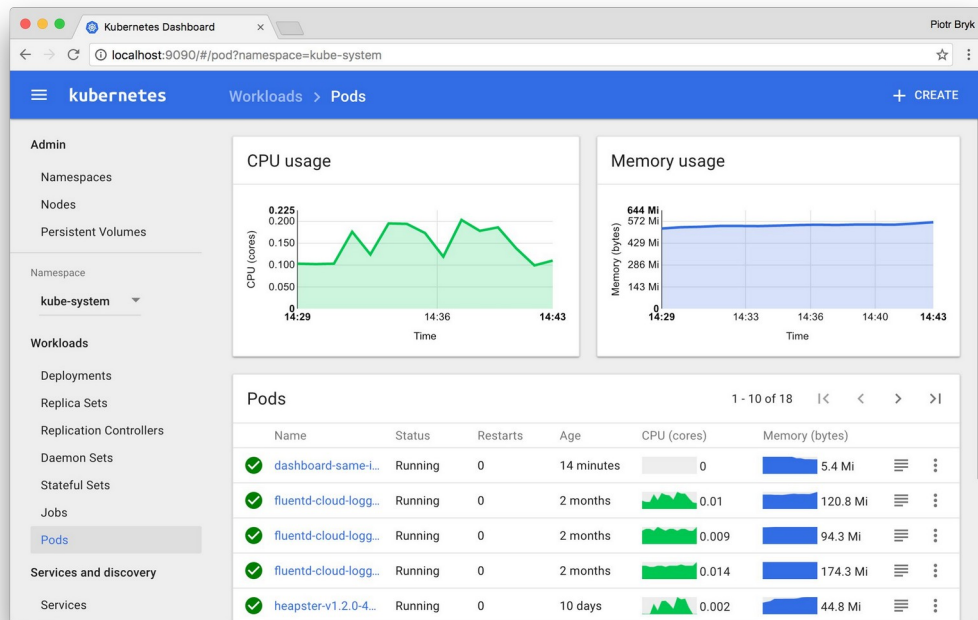


Demo: Services



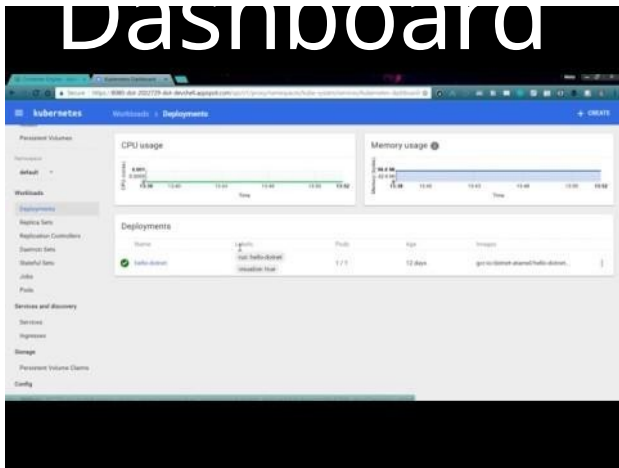
Kubernetes

A general purpose, web-based UI to view/manage Kubernetes clusters



Demo: Kubernetes

Dashboard



Kubernetes Terminology

Deployment

ReplicaSet

DaemonSet

Pod

Liveness Probe

Job

Volume

Readiness Probe

StatefulSet

Label

Service

ConfigMap

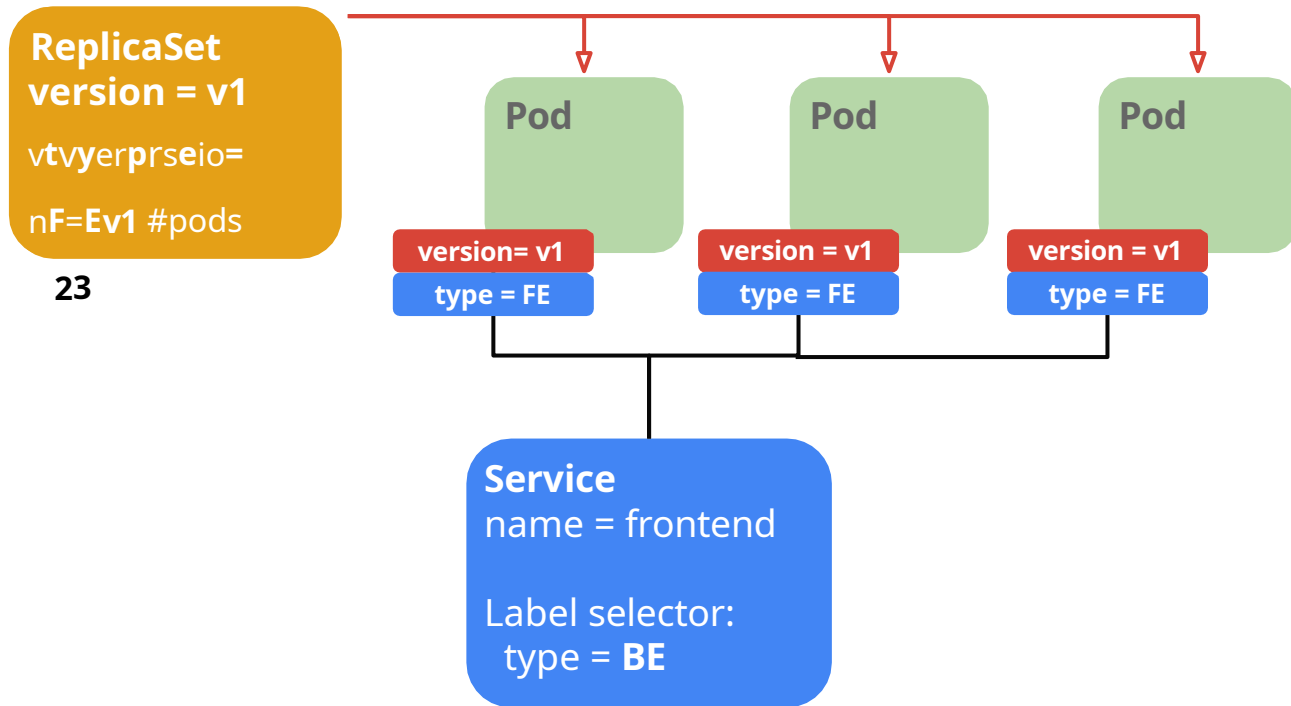
Selector

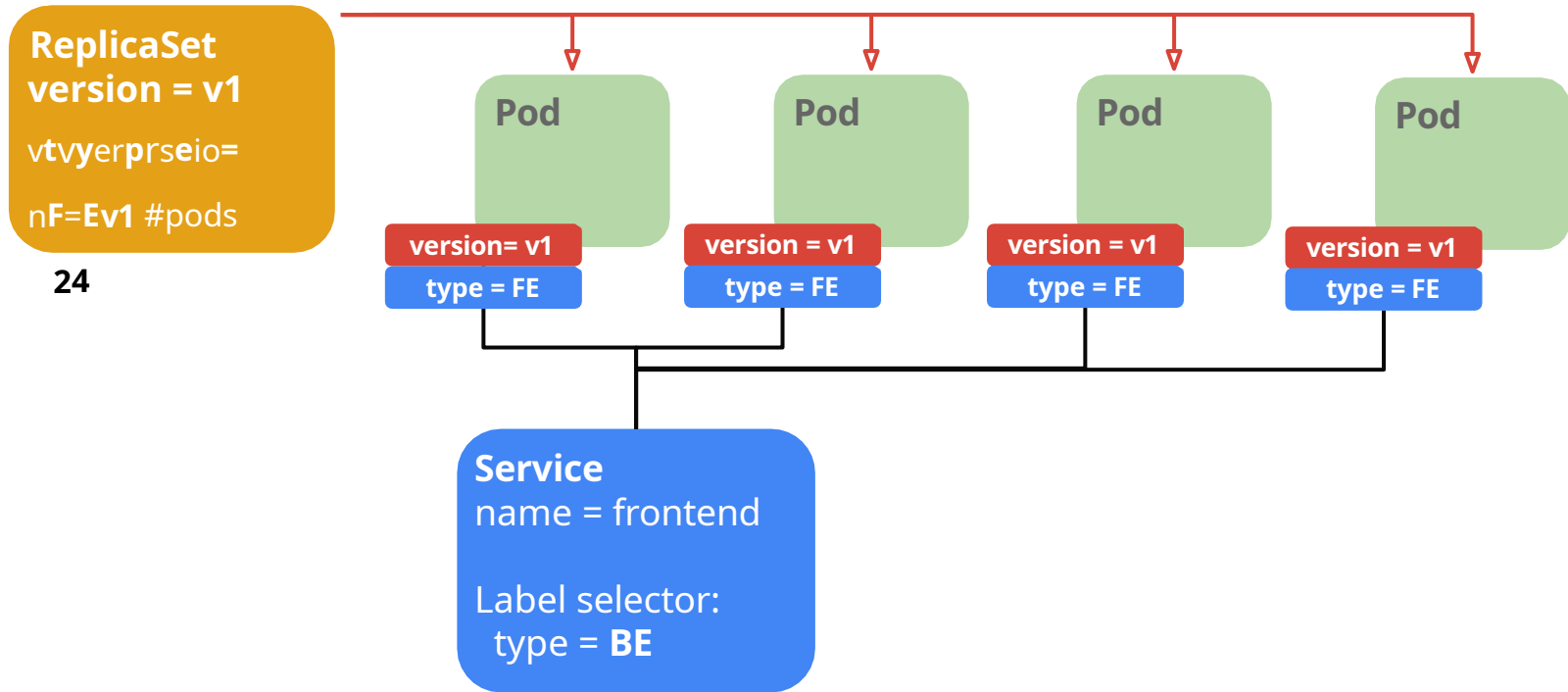
Secret



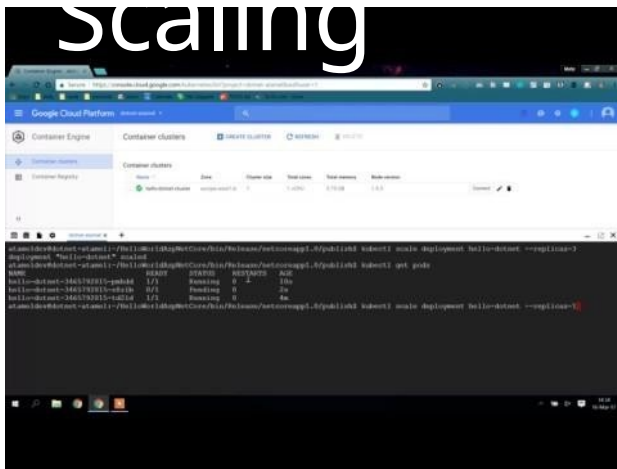
Scalin







Demo: Scaling



Rolling

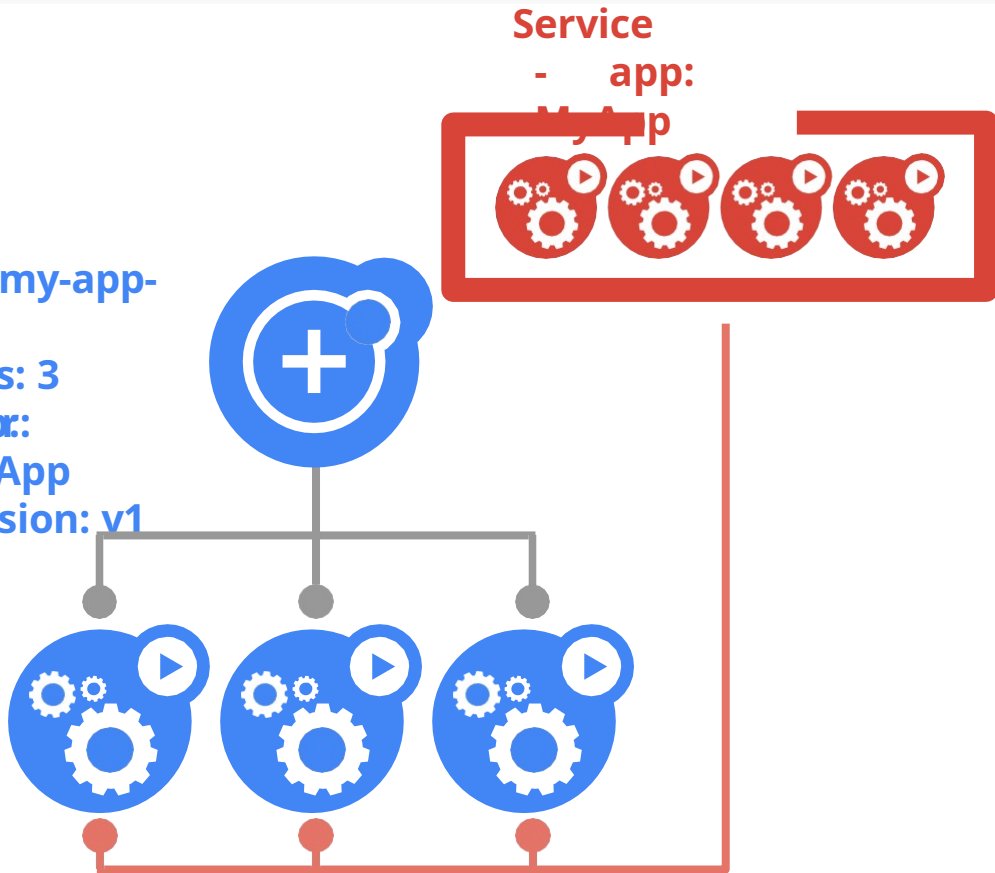


Rolling Update

@meteatame
|

ReplicaSet

- name: my-app-v1
- replicas: 3
- selector:
 - MyApp
- version: v1

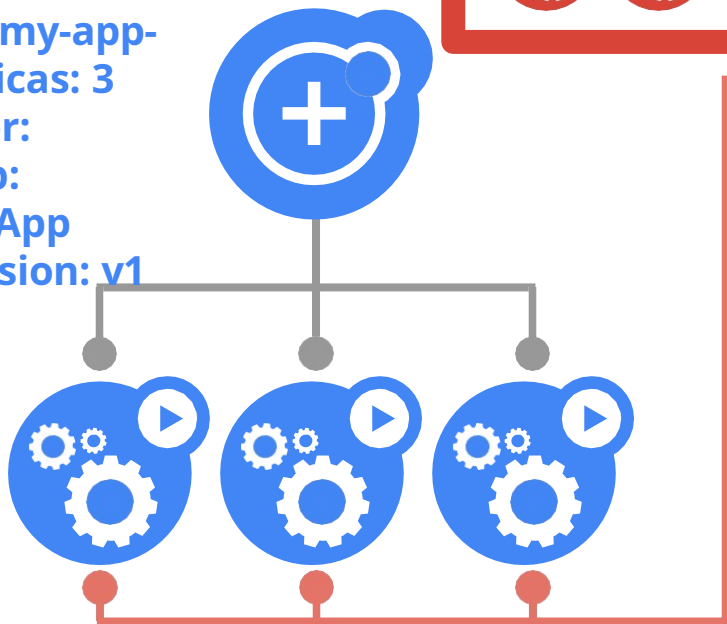


Rolling Update

@meteatame
|

ReplicaSet

- name: my-app-v1
- replicas: 3
- selector:
 - app: MyApp
 - version: v1



ReplicaSet

- name: my-app-v2
- replicas: 0
- selector:
 - app: MyApp
 - version: v2

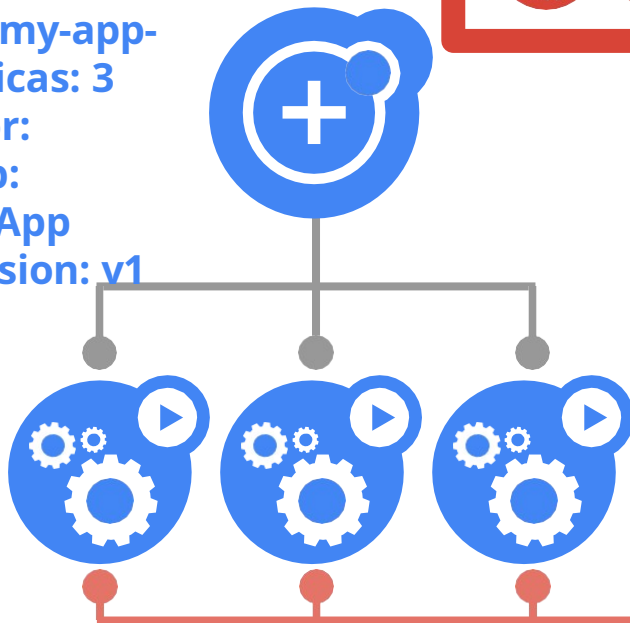


Rolling Update

@meteatame
|

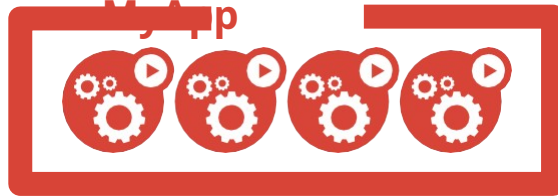
ReplicaSet

- name: my-app-v1
- replicas: 3
- selector:
 - app: MyApp
 - version: v1



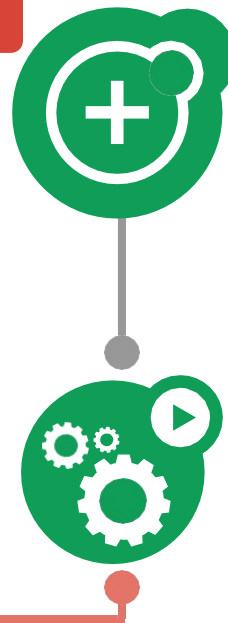
Service

- app: MyApp



ReplicaSet

- name: my-app-v2
- replicas: 1
- selector:
 - app: MyApp
 - version: v2

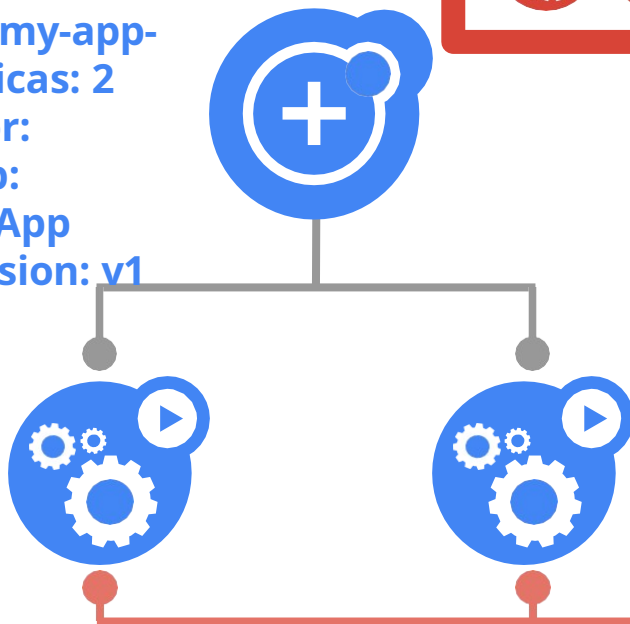


Rolling Update

@meteatame
|

ReplicaSet

- name: my-app-v1
- replicas: 2
- selector:
 - app: MyApp
 - version: v1



ReplicaSet

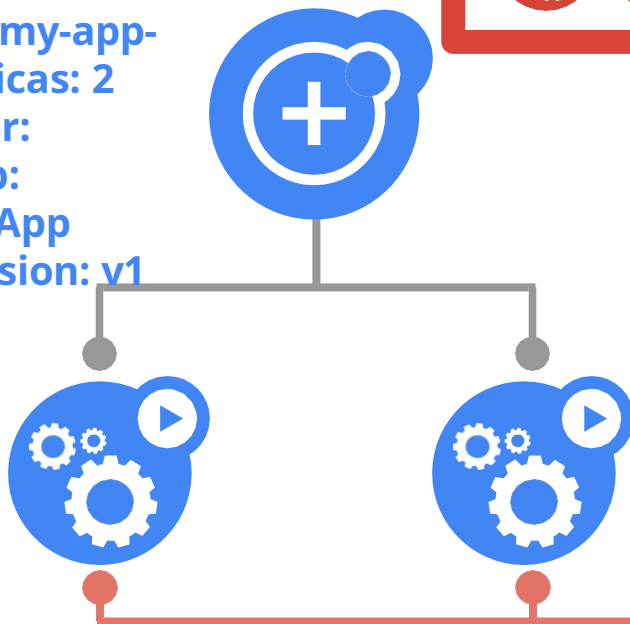
- name: my-app-v2
- replicas: 1
- selector:
 - app: MyApp
 - version: v2

Rolling Update

@meteatame
|

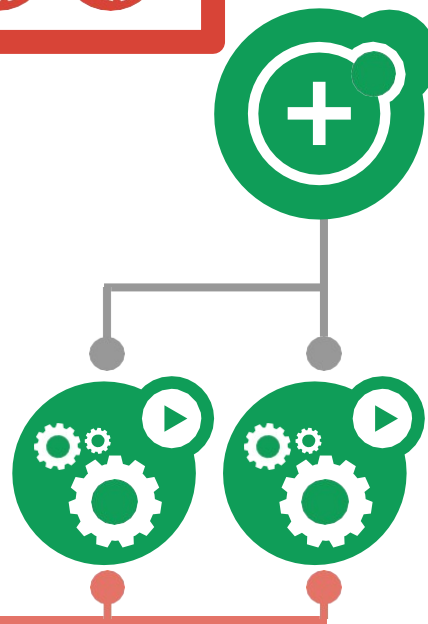
ReplicaSet

- name: my-app-v1
- replicas: 2
- selector:
 - app: MyApp
 - version: v1



ReplicaSet

- name: my-app-v2
- replicas: 2
- selector:
 - app: MyApp
 - version: v2



Rolling Update

@meteatame
|

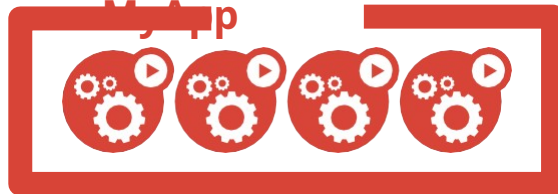
ReplicaSet

- name: my-app-v1
- replicas: 1
- selector:
 - app: MyApp
 - version: v1



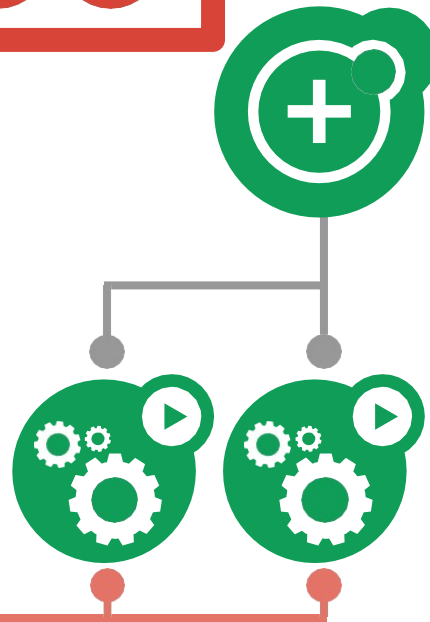
Service

- app: MyApp



ReplicaSet

- name: my-app-v2
- replicas: 2
- selector:
 - app: MyApp
 - version: v2



Rolling Update

@meteatame
|

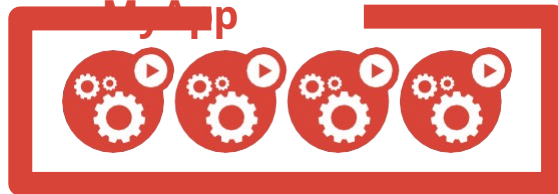
ReplicaSet

- name: my-app-v1
- replicas: 1
- selector:
 - app: MyApp
 - version: v1



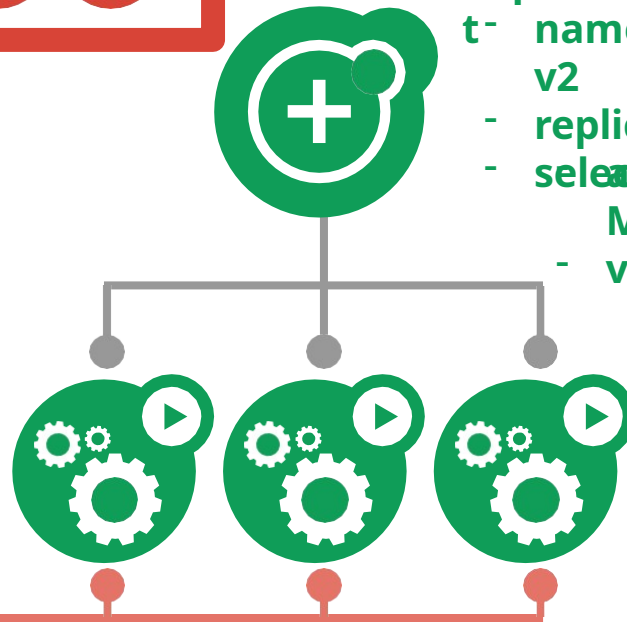
Service

- app: MyApp



ReplicaSet

- name: my-app-v2
- replicas: 3
- selector:
 - app: MyApp
 - version: v2



Rolling Update

@meteatame
|

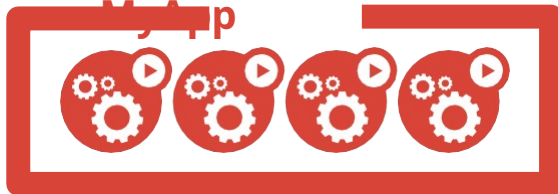
ReplicaSet

- name: my-app-v1
- replicas: 0
- selector:
 - app: MyApp
 - version: v1



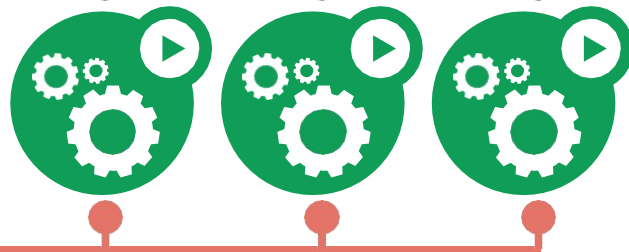
Service

- app: MyApp



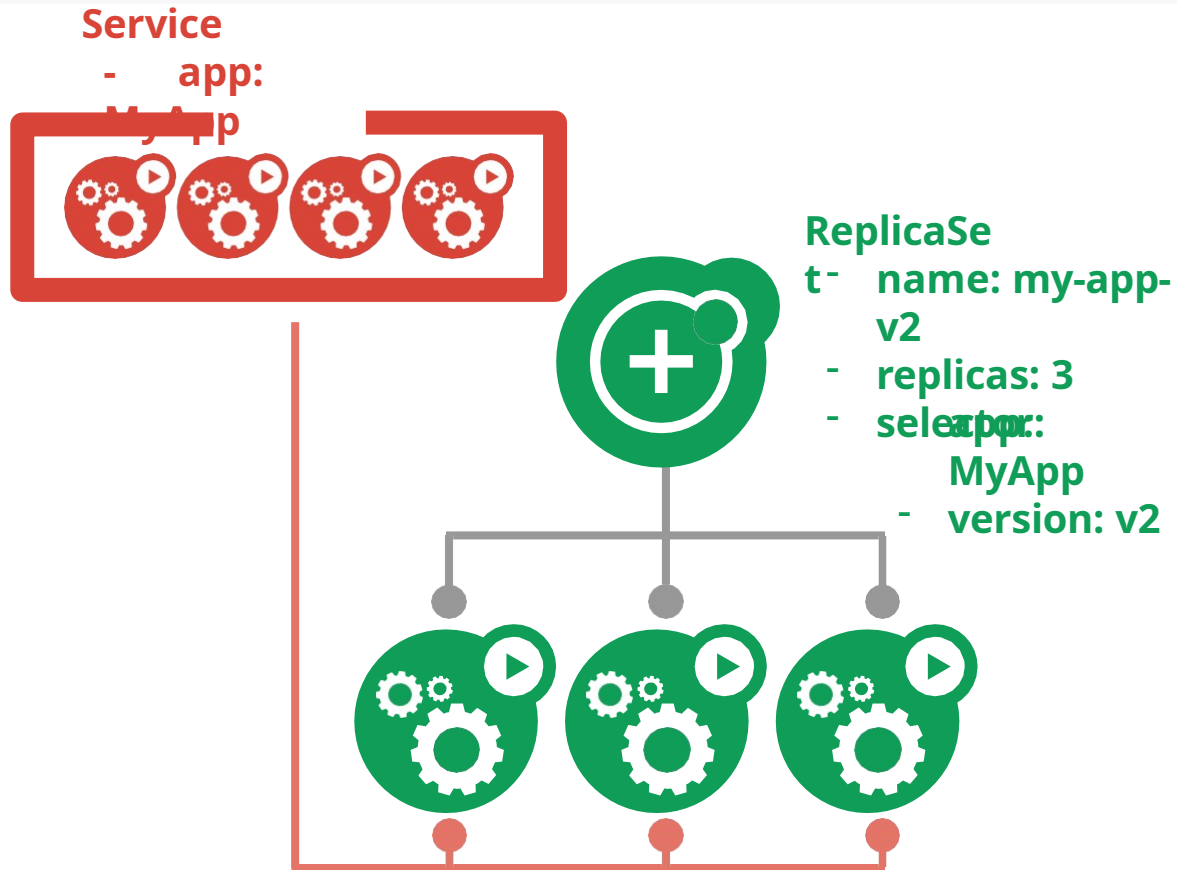
ReplicaSet

- name: my-app-v2
- replicas: 3
- selector:
 - app: MyApp
 - version: v2

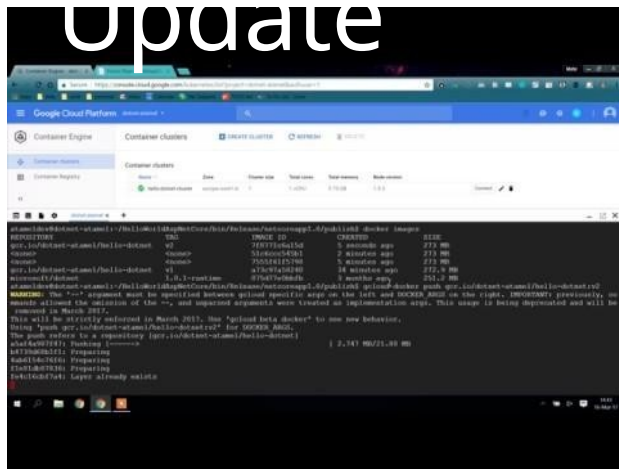


Rolling Update

@meteatame
|

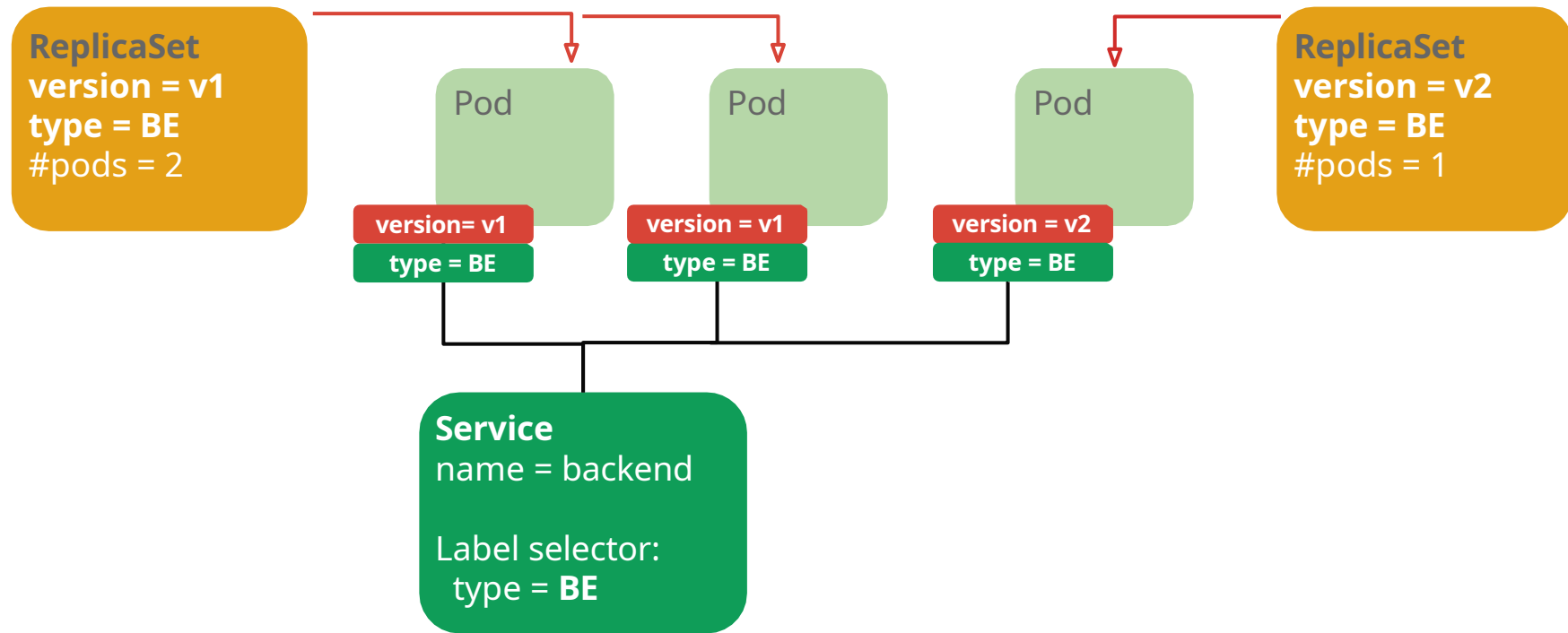


Demo: Rolling Update



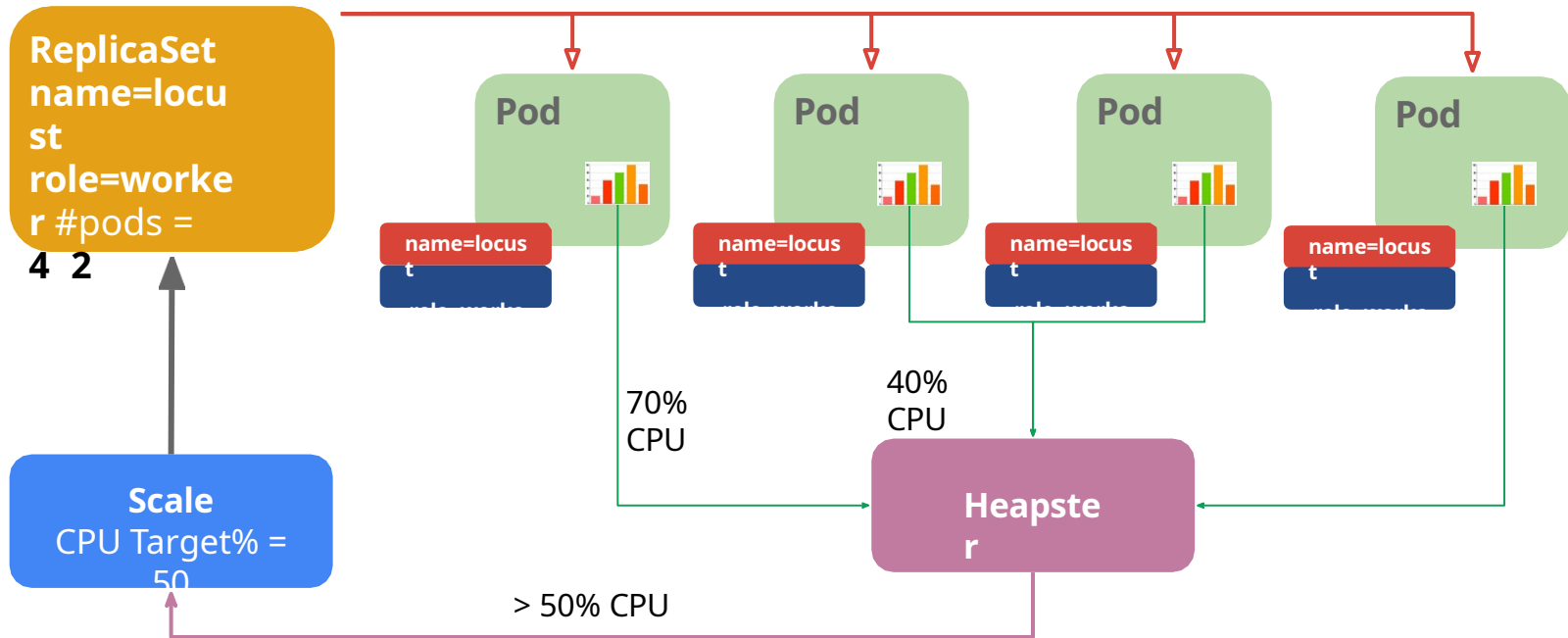
Canary Deployments

@meteatame
|



Autoscaling

@meteatame
|



DaemonSet



Problem: how to run a Pod on every node?

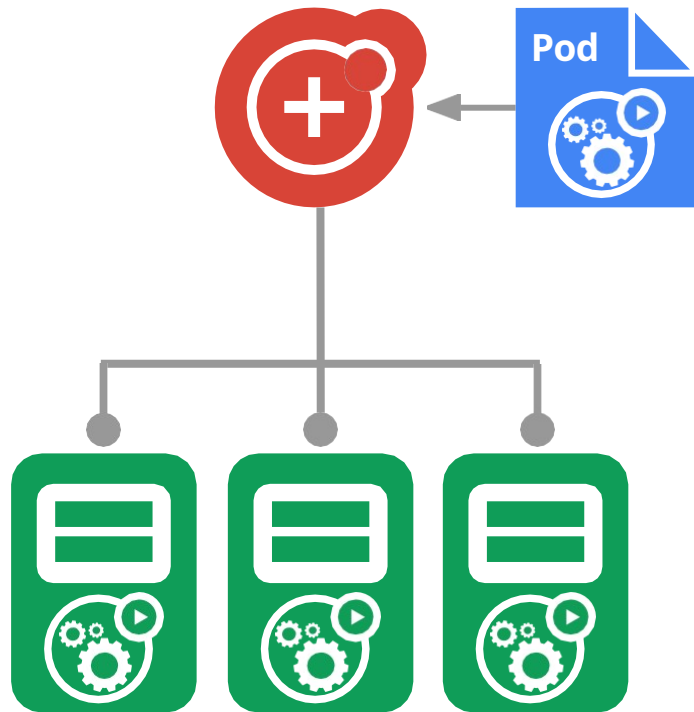
- or a subset of nodes

Similar to ReplicaSet

- principle: do one thing, don't overload

“Which nodes?” is a selector

Use familiar tools and
patterns



Jobs

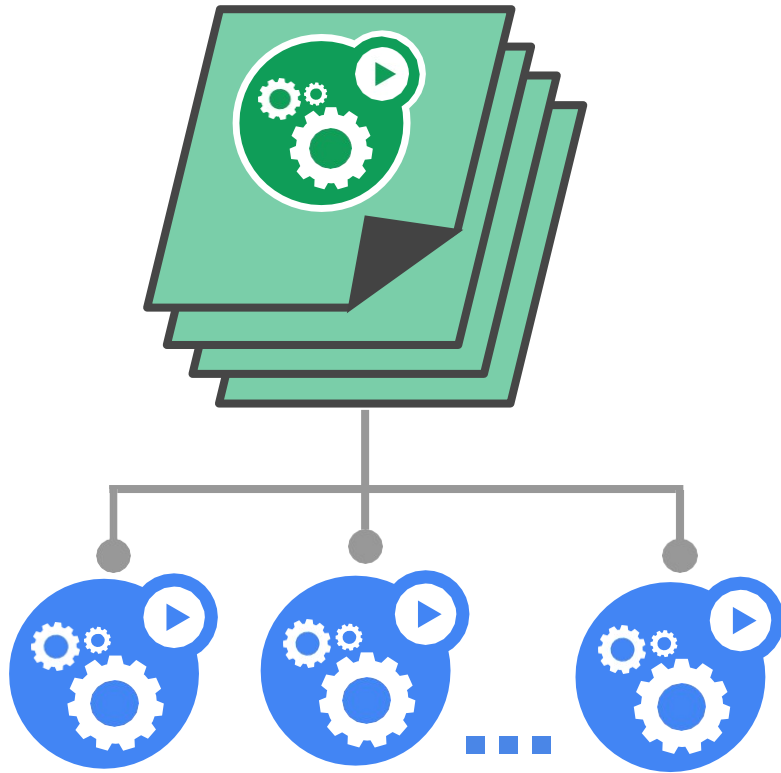


Run-to-completion, as opposed to run-forever

- Express parallelism vs. required completions
- Workflow: restart on failure
- Build/test: don't restart on failure

Aggregates success/failure

counts Built for batch and big-data work



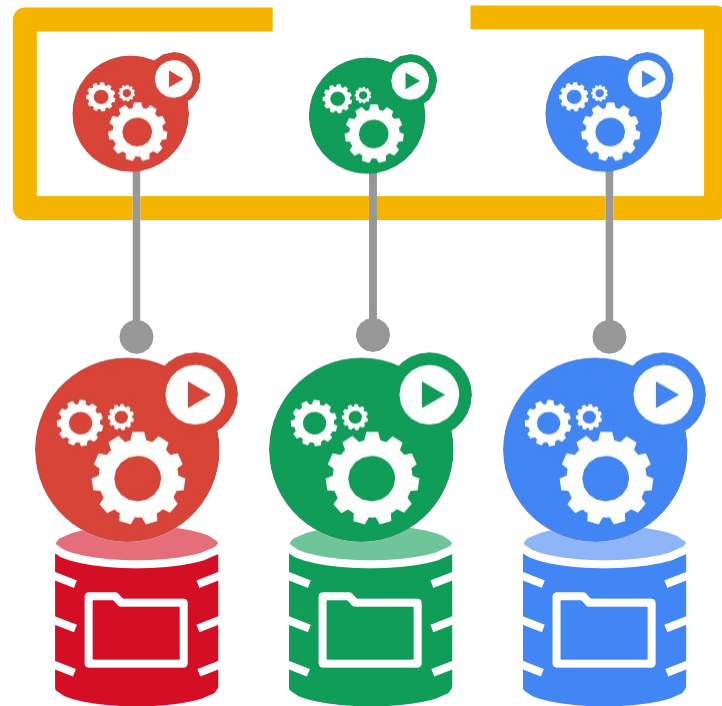
StatefulSet

Goal: enable clustered software on Kubernetes

- mysql, redis, zookeeper, ...

Clustered apps need “identity” and sequencing guarantees

- stable hostname, available in DNS
- an ordinal index
- stable storage: linked to the ordinal & hostname
- discovery of peers for quorum
- startup/teardown ordering



ConfigMap



Goal: manage app configuration

- ...without making overly-brittle container images

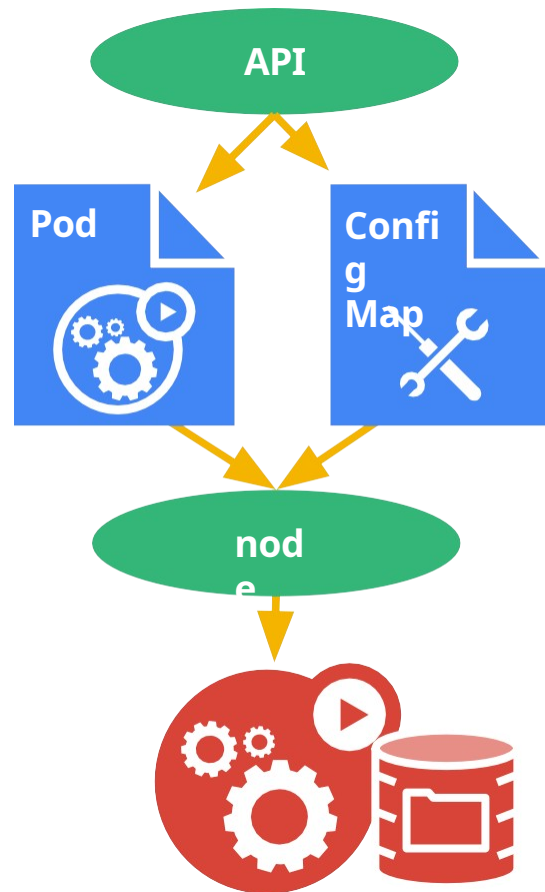
12-factor says config comes from the environment

- Kubernetes is the environment

Manage config via the Kubernetes API

Inject config as a ~~live-updated~~ virtual volume (atomic) into your Pods

- also available as env vars



Secrets



Goal: grant a pod access to a secured *something*

- don't put secrets in the container image!

[12-factor](#) says config comes from the environment

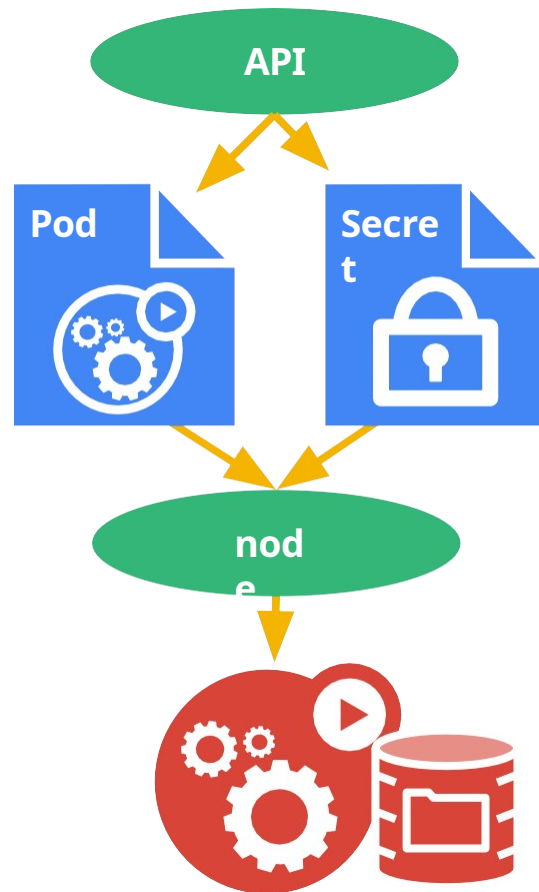
- Kubernetes is the environment

Manage secrets via the Kubernetes API

- late-binding, tmpfs - never touches disk

Inject secrets as virtual volumes into your Pods

- also available as env vars



Kubernetes Terminology

Deployment

ReplicaSet

DaemonSet

Pod

Liveness Probe

Job

Volume

Readiness Probe

StatefulSet

Label

Service

ConfigMap

Selector

Secret



There is
more!



Thank You

kubernetes.i
cloud.google.com/container-
engine

Send talk
feedback
bit.ly/atamel



Mete Atamel
@meteatable
atamel@google.co
m
meteatable.wordpress.com